

V3.0

Generated by Doxygen 1.17.0



|                                                     |          |
|-----------------------------------------------------|----------|
| <b>1 Class</b>                                      | <b>1</b> |
| 1.1 Programos aprašymas                             | 1        |
| 1.2 O1                                              | 1        |
| 1.3 O2                                              | 1        |
| 1.4 O3                                              | 1        |
| <b>2 Hierarchical Index</b>                         | <b>3</b> |
| 2.1 Class Hierarchy                                 | 3        |
| <b>3 Class Index</b>                                | <b>5</b> |
| 3.1 Class List                                      | 5        |
| <b>4 File Index</b>                                 | <b>7</b> |
| 4.1 File List                                       | 7        |
| <b>5 Class Documentation</b>                        | <b>9</b> |
| 5.1 Studentas Class Reference                       | 9        |
| 5.1.1 Detailed Description                          | 10       |
| 5.1.2 Constructor & Destructor Documentation        | 10       |
| 5.1.2.1 Studentas() [1/5]                           | 10       |
| 5.1.2.2 Studentas() [2/5]                           | 10       |
| 5.1.2.3 Studentas() [3/5]                           | 10       |
| 5.1.2.4 Studentas() [4/5]                           | 11       |
| 5.1.2.5 Studentas() [5/5]                           | 11       |
| 5.1.2.6 ~Studentas()                                | 11       |
| 5.1.3 Member Function Documentation                 | 11       |
| 5.1.3.1 egzaminas()                                 | 11       |
| 5.1.3.2 galutBalas()                                | 11       |
| 5.1.3.3 nd()                                        | 11       |
| 5.1.3.4 operator=() [1/2]                           | 11       |
| 5.1.3.5 operator=() [2/2]                           | 12       |
| 5.1.3.6 readStudent()                               | 12       |
| 5.1.3.7 setEgz()                                    | 12       |
| 5.1.3.8 setNd()                                     | 12       |
| 5.1.3.9 spausdinti()                                | 12       |
| 5.1.4 Friends And Related Symbol Documentation      | 12       |
| 5.1.4.1 operator<<                                  | 12       |
| 5.1.4.2 operator>>                                  | 13       |
| 5.2 Vector< T, Allocator > Class Template Reference | 13       |
| 5.2.1 Detailed Description                          | 14       |
| 5.2.2 Member Typedef Documentation                  | 14       |
| 5.2.2.1 allocator_type                              | 14       |
| 5.2.2.2 const_iterator                              | 15       |
| 5.2.2.3 const_pointer                               | 15       |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 5.2.2.4 <a href="#">const_reference</a>                          | 15 |
| 5.2.2.5 <a href="#">const_reverse_iterator</a>                   | 15 |
| 5.2.2.6 <a href="#">difference_type</a>                          | 15 |
| 5.2.2.7 <a href="#">iterator</a>                                 | 15 |
| 5.2.2.8 <a href="#">pointer</a>                                  | 15 |
| 5.2.2.9 <a href="#">reference</a>                                | 16 |
| 5.2.2.10 <a href="#">reverse_iterator</a>                        | 16 |
| 5.2.2.11 <a href="#">size_type</a>                               | 16 |
| 5.2.2.12 <a href="#">value_type</a>                              | 16 |
| 5.2.3 <a href="#">Constructor &amp; Destructor Documentation</a> | 16 |
| 5.2.3.1 <a href="#">Vector() [1/5]</a>                           | 16 |
| 5.2.3.2 <a href="#">Vector() [2/5]</a>                           | 16 |
| 5.2.3.3 <a href="#">Vector() [3/5]</a>                           | 17 |
| 5.2.3.4 <a href="#">~Vector()</a>                                | 17 |
| 5.2.3.5 <a href="#">Vector() [4/5]</a>                           | 17 |
| 5.2.3.6 <a href="#">Vector() [5/5]</a>                           | 17 |
| 5.2.4 <a href="#">Member Function Documentation</a>              | 17 |
| 5.2.4.1 <a href="#">append_range()</a>                           | 17 |
| 5.2.4.2 <a href="#">assign() [1/2]</a>                           | 17 |
| 5.2.4.3 <a href="#">assign() [2/2]</a>                           | 18 |
| 5.2.4.4 <a href="#">assign_range()</a>                           | 18 |
| 5.2.4.5 <a href="#">at() [1/2]</a>                               | 18 |
| 5.2.4.6 <a href="#">at() [2/2]</a>                               | 18 |
| 5.2.4.7 <a href="#">back() [1/2]</a>                             | 18 |
| 5.2.4.8 <a href="#">back() [2/2]</a>                             | 18 |
| 5.2.4.9 <a href="#">begin() [1/2]</a>                            | 19 |
| 5.2.4.10 <a href="#">begin() [2/2]</a>                           | 19 |
| 5.2.4.11 <a href="#">capacity()</a>                              | 19 |
| 5.2.4.12 <a href="#">cbegin()</a>                                | 19 |
| 5.2.4.13 <a href="#">cend()</a>                                  | 19 |
| 5.2.4.14 <a href="#">clear()</a>                                 | 19 |
| 5.2.4.15 <a href="#">crbegin()</a>                               | 19 |
| 5.2.4.16 <a href="#">crend()</a>                                 | 20 |
| 5.2.4.17 <a href="#">data() [1/2]</a>                            | 20 |
| 5.2.4.18 <a href="#">data() [2/2]</a>                            | 20 |
| 5.2.4.19 <a href="#">emplace()</a>                               | 20 |
| 5.2.4.20 <a href="#">emplace_back()</a>                          | 20 |
| 5.2.4.21 <a href="#">empty()</a>                                 | 20 |
| 5.2.4.22 <a href="#">end() [1/2]</a>                             | 21 |
| 5.2.4.23 <a href="#">end() [2/2]</a>                             | 21 |
| 5.2.4.24 <a href="#">erase() [1/2]</a>                           | 21 |
| 5.2.4.25 <a href="#">erase() [2/2]</a>                           | 21 |

|                                              |           |
|----------------------------------------------|-----------|
| 5.2.4.26 front() [1/2]                       | 21        |
| 5.2.4.27 front() [2/2]                       | 21        |
| 5.2.4.28 get_allocator()                     | 22        |
| 5.2.4.29 insert() [1/2]                      | 22        |
| 5.2.4.30 insert() [2/2]                      | 22        |
| 5.2.4.31 max_size()                          | 22        |
| 5.2.4.32 operator=() [1/2]                   | 22        |
| 5.2.4.33 operator=() [2/2]                   | 22        |
| 5.2.4.34 operator[]() [1/2]                  | 23        |
| 5.2.4.35 operator[]() [2/2]                  | 23        |
| 5.2.4.36 pop_back()                          | 23        |
| 5.2.4.37 push_back()                         | 23        |
| 5.2.4.38 rbegin()                            | 23        |
| 5.2.4.39 rend()                              | 23        |
| 5.2.4.40 reserve()                           | 24        |
| 5.2.4.41 resize()                            | 24        |
| 5.2.4.42 shrink_to_fit()                     | 24        |
| 5.2.4.43 size()                              | 24        |
| 5.2.4.44 swap()                              | 24        |
| 5.3 Zmogus Class Reference                   | 24        |
| 5.3.1 Detailed Description                   | 25        |
| 5.3.2 Constructor & Destructor Documentation | 25        |
| 5.3.2.1 Zmogus() [1/2]                       | 25        |
| 5.3.2.2 Zmogus() [2/2]                       | 25        |
| 5.3.2.3 ~Zmogus()                            | 25        |
| 5.3.3 Member Function Documentation          | 25        |
| 5.3.3.1 pavarde()                            | 25        |
| 5.3.3.2 setPavarde()                         | 26        |
| 5.3.3.3 setVardas()                          | 26        |
| 5.3.3.4 spausdinti()                         | 26        |
| 5.3.3.5 vardas()                             | 26        |
| 5.3.4 Member Data Documentation              | 26        |
| 5.3.4.1 pavarde_                             | 26        |
| 5.3.4.2 vardas_                              | 26        |
| <b>6 File Documentation</b>                  | <b>27</b> |
| 6.1 funkcijos.cpp File Reference             | 27        |
| 6.1.1 Typedef Documentation                  | 28        |
| 6.1.1.1 int_dis                              | 28        |
| 6.1.1.2 laik                                 | 28        |
| 6.1.2 Function Documentation                 | 29        |
| 6.1.2.1 atminties_testas()                   | 29        |

|                                 |    |
|---------------------------------|----|
| 6.1.2.2 check()                 | 29 |
| 6.1.2.3 failu_kurimas()         | 29 |
| 6.1.2.4 gen_pazym()             | 29 |
| 6.1.2.5 gen_vrd()               | 29 |
| 6.1.2.6 isved()                 | 29 |
| 6.1.2.7 isvedimas_faila() [1/3] | 29 |
| 6.1.2.8 isvedimas_faila() [2/3] | 30 |
| 6.1.2.9 isvedimas_faila() [3/3] | 30 |
| 6.1.2.10 isvedimas_faila_visi() | 30 |
| 6.1.2.11 laiko_test()           | 30 |
| 6.1.2.12 pasirink()             | 30 |
| 6.1.2.13 rikiav() [1/3]         | 30 |
| 6.1.2.14 rikiav() [2/3]         | 31 |
| 6.1.2.15 rikiav() [3/3]         | 31 |
| 6.1.2.16 rikiav_visi()          | 31 |
| 6.1.2.17 rusiavimas() [1/3]     | 31 |
| 6.1.2.18 rusiavimas() [2/3]     | 31 |
| 6.1.2.19 rusiavimas() [3/3]     | 31 |
| 6.1.2.20 rusiavimas_visi()      | 32 |
| 6.1.2.21 skaiciai()             | 32 |
| 6.1.2.22 skt() [1/3]            | 32 |
| 6.1.2.23 skt() [2/3]            | 32 |
| 6.1.2.24 skt() [3/3]            | 32 |
| 6.1.2.25 skt_visi()             | 32 |
| 6.1.2.26 strat2() [1/3]         | 33 |
| 6.1.2.27 strat2() [2/3]         | 33 |
| 6.1.2.28 strat2() [3/3]         | 33 |
| 6.1.2.29 strat2_visi()          | 33 |
| 6.1.2.30 strat3() [1/3]         | 33 |
| 6.1.2.31 strat3() [2/3]         | 33 |
| 6.1.2.32 strat3() [3/3]         | 34 |
| 6.1.2.33 strat3_visi()          | 34 |
| 6.1.2.34 testavimas()           | 34 |
| 6.1.2.35 tyrimas1()             | 34 |
| 6.1.2.36 tyrimas2()             | 34 |
| 6.1.2.37 tyrimas2_de()          | 34 |
| 6.1.2.38 tyrimas2_li()          | 34 |
| 6.1.2.39 tyrimas2_ve()          | 35 |
| 6.1.2.40 tyrimas2_visi()        | 35 |
| 6.2 funkcijos.cpp               | 35 |
| 6.3 funkcijos.h File Reference  | 41 |
| 6.3.1 Function Documentation    | 42 |

|                                 |    |
|---------------------------------|----|
| 6.3.1.1 atminties_testas()      | 42 |
| 6.3.1.2 check()                 | 43 |
| 6.3.1.3 failu_kurimas()         | 43 |
| 6.3.1.4 gen_pazym()             | 43 |
| 6.3.1.5 gen_vrd()               | 43 |
| 6.3.1.6 isved()                 | 43 |
| 6.3.1.7 isvedimas_faila() [1/3] | 43 |
| 6.3.1.8 isvedimas_faila() [2/3] | 43 |
| 6.3.1.9 isvedimas_faila() [3/3] | 44 |
| 6.3.1.10 laiko_test()           | 44 |
| 6.3.1.11 pasirink()             | 44 |
| 6.3.1.12 rikiav() [1/3]         | 44 |
| 6.3.1.13 rikiav() [2/3]         | 44 |
| 6.3.1.14 rikiav() [3/3]         | 44 |
| 6.3.1.15 rusiavimas() [1/3]     | 44 |
| 6.3.1.16 rusiavimas() [2/3]     | 45 |
| 6.3.1.17 rusiavimas() [3/3]     | 45 |
| 6.3.1.18 skaiciai()             | 45 |
| 6.3.1.19 skt() [1/3]            | 45 |
| 6.3.1.20 skt() [2/3]            | 45 |
| 6.3.1.21 skt() [3/3]            | 45 |
| 6.3.1.22 strat2() [1/3]         | 46 |
| 6.3.1.23 strat2() [2/3]         | 46 |
| 6.3.1.24 strat2() [3/3]         | 46 |
| 6.3.1.25 strat3() [1/3]         | 46 |
| 6.3.1.26 strat3() [2/3]         | 46 |
| 6.3.1.27 strat3() [3/3]         | 46 |
| 6.3.1.28 testavimas()           | 47 |
| 6.3.1.29 tyrimas1()             | 47 |
| 6.3.1.30 tyrimas2()             | 47 |
| 6.3.1.31 tyrimas2_de()          | 47 |
| 6.3.1.32 tyrimas2_li()          | 47 |
| 6.3.1.33 tyrimas2_ve()          | 47 |
| 6.4 funkcijos.h                 | 48 |
| 6.5 header.h File Reference     | 48 |
| 6.5.1 Function Documentation    | 49 |
| 6.5.1.1 compare()               | 49 |
| 6.5.1.2 comparePagalEgza()      | 49 |
| 6.5.1.3 comparePagalPavarde()   | 49 |
| 6.5.1.4 mediana()               | 49 |
| 6.5.1.5 vidurkis()              | 50 |
| 6.6 header.h                    | 50 |

---

|                                            |           |
|--------------------------------------------|-----------|
| 6.7 README.md File Reference . . . . .     | 51        |
| 6.8 studentas.cpp File Reference . . . . . | 51        |
| 6.8.1 Function Documentation . . . . .     | 51        |
| 6.8.1.1 compare() . . . . .                | 51        |
| 6.8.1.2 comparePagalEgza() . . . . .       | 51        |
| 6.8.1.3 comparePagalPavarde() . . . . .    | 52        |
| 6.8.1.4 mediana() . . . . .                | 52        |
| 6.8.1.5 operator<<() . . . . .             | 52        |
| 6.8.1.6 operator>>() . . . . .             | 52        |
| 6.8.1.7 vidurkis() . . . . .               | 52        |
| 6.9 studentas.cpp . . . . .                | 53        |
| 6.10 vec.h File Reference . . . . .        | 54        |
| 6.10.1 Function Documentation . . . . .    | 55        |
| 6.10.1.1 operator"!=() . . . . .           | 55        |
| 6.10.1.2 operator<() . . . . .             | 55        |
| 6.10.1.3 operator<=() . . . . .            | 55        |
| 6.10.1.4 operator==() . . . . .            | 56        |
| 6.10.1.5 operator>() . . . . .             | 56        |
| 6.10.1.6 operator>=() . . . . .            | 56        |
| 6.10.1.7 swap() . . . . .                  | 56        |
| 6.11 vec.h . . . . .                       | 56        |
| 6.12 vector.cpp File Reference . . . . .   | 60        |
| 6.12.1 Typedef Documentation . . . . .     | 61        |
| 6.12.1.1 int_dis . . . . .                 | 61        |
| 6.12.2 Function Documentation . . . . .    | 61        |
| 6.12.2.1 main() . . . . .                  | 61        |
| 6.13 vector.cpp . . . . .                  | 62        |
| <b>Index</b>                               | <b>67</b> |



# Chapter 1

## Class

### 1.1 Programos aprašymas

Ši programa turi galimybę nuskaityti duomenis įvestus vartotojo arba nuskaityti tiesiai iš failo bei generuoti savo duomenis. Programoje yra realizuoti visi Rule of five operatoriai: copy constructor, move constructor, copy assignment, move assignment, destructor. Taip pat realizuoti ir įvesties bei išvesties operatoriai. Šioje programos versijoje programa jau naudoja klasę - **Žmogus** ir jos išvestinę klasę - **Studentas**.

### 1.2 O1

| Konteineris            | Strategija | 100k įrašų | 1M įrašų | exe failo dydis |
|------------------------|------------|------------|----------|-----------------|
| <a href="#">Vector</a> | 1          | 0,04       | 0,3      | 294 KB          |
| <a href="#">Vector</a> | 2          | 0,05       | 0,4      | 294 KB          |
| <a href="#">Vector</a> | 3          | 0,03       | 0,3      | 294 KB          |

### 1.3 O2

| Konteineris            | Strategija | 100k įrašų | 1M įrašų | exe failo dydis |
|------------------------|------------|------------|----------|-----------------|
| <a href="#">Vector</a> | 1          | 0,03       | 0,3      | 280 KB          |
| <a href="#">Vector</a> | 2          | 0,03       | 0,3      | 280 KB          |
| <a href="#">Vector</a> | 3          | 0,03       | 0,2      | 280 KB          |

### 1.4 O3

| Konteineris            | Strategija | 100k įrašų | 1M įrašų | exe failo dydis |
|------------------------|------------|------------|----------|-----------------|
| <a href="#">Vector</a> | 1          | 0,04       | 0,5      | 289 KB          |
| <a href="#">Vector</a> | 2          | 0,05       | 0,5      | 289 KB          |

| Konteineris | Strategija | 100k įrašų | 1M įrašų | exe failo dydis |
|-------------|------------|------------|----------|-----------------|
| Vector      | 3          | 0,03       | 0,4      | 289 KB          |

## Chapter 2

# Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                                  |    |
|----------------------------------|----|
| Vector< T, Allocator > . . . . . | 13 |
| Zmogus . . . . .                 | 24 |
| Studentas . . . . .              | 9  |



## Chapter 3

# Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                        |    |
|--------------------------------------------------------|----|
| <a href="#">Studentas</a> . . . . .                    | 9  |
| <a href="#">Vector&lt; T, Allocator &gt;</a> . . . . . | 13 |
| <a href="#">Zmogus</a> . . . . .                       | 24 |



# Chapter 4

## File Index

### 4.1 File List

Here is a list of all files with brief descriptions:

|                               |    |
|-------------------------------|----|
| <a href="#">funkcijos.cpp</a> | 27 |
| <a href="#">funkcijos.h</a>   | 41 |
| <a href="#">header.h</a>      | 48 |
| <a href="#">studentas.cpp</a> | 51 |
| <a href="#">vec.h</a>         | 54 |
| <a href="#">vector.cpp</a>    | 60 |





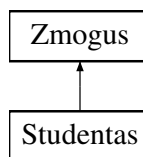
# Chapter 5

## Class Documentation

### 5.1 Studentas Class Reference

```
#include <header.h>
```

Inheritance diagram for Studentas:



#### Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (std::istream &is)
- [Studentas](#) (const std::string &vardas, const std::string &pavarde, int egzaminas, const [Vector](#)< double > &nd)
- int [egzaminas](#) () const
- [Vector](#)< double > [nd](#) () const
- double [galutiBalas](#) (double(\*f)([Vector](#)< double >)=[mediana](#)) const
- std::istream & [readStudent](#) (std::istream &is)
- void [setEgz](#) (double egzam)
- void [setNd](#) (double x)
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other) noexcept
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- void [spausdinti](#) () const override
- [~Studentas](#) ()

## Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual std::string [vardas](#) () const
- virtual std::string [pavarde](#) () const
- virtual void [setVardas](#) (std::string &v)
- virtual void [setPavarde](#) (std::string &p)
- virtual [~Zmogus](#) ()

## Friends

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)

## Additional Inherited Members

## Protected Attributes inherited from [Zmogus](#)

- std::string [vardas\\_](#)
- std::string [pavarde\\_](#)

### 5.1.1 Detailed Description

Definition at line 34 of file [header.h](#).

### 5.1.2 Constructor & Destructor Documentation

#### 5.1.2.1 [Studentas\(\)](#) [1/5]

```
Studentas::Studentas () [inline]
```

Definition at line 39 of file [header.h](#).

#### 5.1.2.2 [Studentas\(\)](#) [2/5]

```
Studentas::Studentas (
    std::istream & is)
```

Definition at line 13 of file [studentas.cpp](#).

#### 5.1.2.3 [Studentas\(\)](#) [3/5]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    int egzaminas,
    const Vector< double > & nd)
```

Definition at line 22 of file [studentas.cpp](#).

#### 5.1.2.4 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & other)
```

Definition at line 99 of file [studentas.cpp](#).

#### 5.1.2.5 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Definition at line 106 of file [studentas.cpp](#).

#### 5.1.2.6 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Definition at line 68 of file [header.h](#).

### 5.1.3 Member Function Documentation

#### 5.1.3.1 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

Definition at line 44 of file [header.h](#).

#### 5.1.3.2 galutBalas()

```
double Studentas::galutBalas (  
    double(* f) (Vector< double >) = mediana) const
```

Definition at line 18 of file [studentas.cpp](#).

#### 5.1.3.3 nd()

```
Vector< double > Studentas::nd () const [inline]
```

Definition at line 45 of file [header.h](#).

#### 5.1.3.4 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Definition at line 115 of file [studentas.cpp](#).

#### 5.1.3.5 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Definition at line 128 of file [studentas.cpp](#).

#### 5.1.3.6 readStudent()

```
std::istream & Studentas::readStudent (  
    std::istream & is)
```

Definition at line 29 of file [studentas.cpp](#).

#### 5.1.3.7 setEgz()

```
void Studentas::setEgz (  
    double egzam) [inline]
```

Definition at line 50 of file [header.h](#).

#### 5.1.3.8 setNd()

```
void Studentas::setNd (  
    double x) [inline]
```

Definition at line 51 of file [header.h](#).

#### 5.1.3.9 spausdinti()

```
void Studentas::spausdinti () const [override], [virtual]
```

Implements [Zmogus](#).

Definition at line 139 of file [studentas.cpp](#).

### 5.1.4 Friends And Related Symbol Documentation

#### 5.1.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s) [friend]
```

Definition at line 53 of file [studentas.cpp](#).

## 5.1.4.2 operator&gt;&gt;

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Definition at line 49 of file [studentas.cpp](#).

The documentation for this class was generated from the following files:

- [header.h](#)
- [studentas.cpp](#)

## 5.2 Vector&lt; T, Allocator &gt; Class Template Reference

```
#include <vec.h>
```

## Public Types

- using [value\\_type](#) = T
- using [allocator\\_type](#) = Allocator
- using [size\\_type](#) = std::size\_t
- using [difference\\_type](#) = std::ptrdiff\_t
- using [reference](#) = [value\\_type](#)&
- using [const\\_reference](#) = const [value\\_type](#)&
- using [pointer](#) = typename std::allocator\_traits<Allocator>::pointer
- using [const\\_pointer](#) = typename std::allocator\_traits<Allocator>::const\_pointer
- using [iterator](#) = T\*
- using [const\\_iterator](#) = const T\*
- using [reverse\\_iterator](#) = std::reverse\_iterator<[iterator](#)>
- using [const\\_reverse\\_iterator](#) = std::reverse\_iterator<[const\\_iterator](#)>

## Public Member Functions

- [Vector](#) ()
- [Vector](#) ([size\\_type](#) n, const T &value=T())
- [Vector](#) (std::initializer\_list< T > init)
- [~Vector](#) ()
- [Vector](#) (const [Vector](#) &other)
- [Vector](#) ([Vector](#) &&other) noexcept
- [Vector](#) & [operator=](#) (const [Vector](#) &other)
- [Vector](#) & [operator=](#) ([Vector](#) &&other) noexcept
- void [assign](#) ([size\\_type](#) count, const T &value)
- template<typename InputIt>  
std::enable\_if\_t<!std::is\_integral< InputIt >::value, void > [assign](#) (InputIt first, InputIt last)
- template<typename Range>  
void [assign\\_range](#) (const Range &r)
- std::allocator< T > [get\\_allocator](#) () const
- [reference](#) at ([size\\_type](#) i)
- [const\\_reference](#) at ([size\\_type](#) i) const

- [reference operator\[\]](#) ([size\\_type](#) i)
- [const\\_reference operator\[\]](#) ([size\\_type](#) i) const
- [reference front](#) ()
- [const\\_reference front](#) () const
- [reference back](#) ()
- [const\\_reference back](#) () const
- [T \\* data](#) ()
- [const T \\* data](#) () const
- [iterator begin](#) ()
- [const\\_iterator begin](#) () const
- [iterator end](#) ()
- [const\\_iterator end](#) () const
- [const\\_iterator cbegin](#) () const
- [const\\_iterator cend](#) () const
- [reverse\\_iterator rbegin](#) ()
- [reverse\\_iterator rend](#) ()
- [const\\_reverse\\_iterator crbegin](#) () const
- [const\\_reverse\\_iterator crend](#) () const
- [bool empty](#) () const
- [size\\_type size](#) () const
- [size\\_type max\\_size](#) () const
- [void reserve](#) ([size\\_type](#) new\_cap)
- [size\\_type capacity](#) () const
- [void shrink\\_to\\_fit](#) ()
- [void clear](#) ()
- [iterator insert](#) ([iterator](#) pos, const T &value)
- [iterator insert](#) ([iterator](#) pos, [size\\_type](#) count, const T &value)
- [template<typename... Args>](#)  
  [iterator emplace](#) ([iterator](#) pos, Args &&... args)
- [iterator erase](#) ([iterator](#) pos)
- [iterator erase](#) ([iterator](#) first, [iterator](#) last)
- [void push\\_back](#) (const T &value)
- [template<typename... Args>](#)  
  [reference emplace\\_back](#) (Args &&... args)
- [template<typename Range>](#)  
  [void append\\_range](#) (const Range &r)
- [void pop\\_back](#) ()
- [void resize](#) ([size\\_type](#) new\_size)
- [void swap](#) ([Vector](#) &other) noexcept

### 5.2.1 Detailed Description

```
template<class T, class Allocator = std::allocator<T>>
class Vector< T, Allocator >
```

Definition at line 12 of file [vec.h](#).

### 5.2.2 Member Typedef Documentation

#### 5.2.2.1 allocator\_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::allocator_type = Allocator
```

Definition at line 16 of file [vec.h](#).

### 5.2.2.2 const\_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const T*
```

Definition at line 24 of file [vec.h](#).

### 5.2.2.3 const\_pointer

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_pointer = typename std::allocator_traits<Allocator>::const_pointer
```

Definition at line 22 of file [vec.h](#).

### 5.2.2.4 const\_reference

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const value_type&
```

Definition at line 20 of file [vec.h](#).

### 5.2.2.5 const\_reverse\_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Definition at line 26 of file [vec.h](#).

### 5.2.2.6 difference\_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Definition at line 18 of file [vec.h](#).

### 5.2.2.7 iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::iterator = T*
```

Definition at line 23 of file [vec.h](#).

### 5.2.2.8 pointer

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::pointer = typename std::allocator_traits<Allocator>::pointer
```

Definition at line 21 of file [vec.h](#).

#### 5.2.2.9 reference

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::reference = value_type&
```

Definition at line 19 of file [vec.h](#).

#### 5.2.2.10 reverse\_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

Definition at line 25 of file [vec.h](#).

#### 5.2.2.11 size\_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::size_type = std::size_t
```

Definition at line 17 of file [vec.h](#).

#### 5.2.2.12 value\_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::value_type = T
```

Definition at line 15 of file [vec.h](#).

### 5.2.3 Constructor & Destructor Documentation

#### 5.2.3.1 Vector() [1/5]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline]
```

Definition at line 43 of file [vec.h](#).

#### 5.2.3.2 Vector() [2/5]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type n,
    const T & value = T()) [inline], [explicit]
```

Definition at line 45 of file [vec.h](#).



**5.2.3.3 Vector()** [3/5]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > init) [inline]
```

Definition at line 50 of file [vec.h](#).

**5.2.3.4 ~Vector()**

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector () [inline]
```

Definition at line 57 of file [vec.h](#).

**5.2.3.5 Vector()** [4/5]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Definition at line 59 of file [vec.h](#).

**5.2.3.6 Vector()** [5/5]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Definition at line 66 of file [vec.h](#).

**5.2.4 Member Function Documentation****5.2.4.1 append\_range()**

```
template<class T, class Allocator = std::allocator<T>>
template<typename Range>
void Vector< T, Allocator >::append_range (
    const Range & r) [inline]
```

Definition at line 277 of file [vec.h](#).

**5.2.4.2 assign()** [1/2]

```
template<class T, class Allocator = std::allocator<T>>
template<typename InputIt>
std::enable_if_t<!std::is_integral< InputIt >::value, void > Vector< T, Allocator >::assign
(
    InputIt first,
    InputIt last) [inline]
```

Definition at line 112 of file [vec.h](#).

#### 5.2.4.3 `assign()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type count,
    const T & value) [inline]
```

Definition at line 100 of file [vec.h](#).

#### 5.2.4.4 `assign_range()`

```
template<class T, class Allocator = std::allocator<T>>
template<typename Range>
void Vector< T, Allocator >::assign_range (
    const Range & r) [inline]
```

Definition at line 120 of file [vec.h](#).

#### 5.2.4.5 `at()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type i) [inline]
```

Definition at line 133 of file [vec.h](#).

#### 5.2.4.6 `at()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type i) const [inline]
```

Definition at line 138 of file [vec.h](#).

#### 5.2.4.7 `back()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back () [inline]
```

Definition at line 149 of file [vec.h](#).

#### 5.2.4.8 `back()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back () const [inline]
```

Definition at line 150 of file [vec.h](#).

#### 5.2.4.9 begin() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin () [inline]
```

Definition at line 158 of file [vec.h](#).

#### 5.2.4.10 begin() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin () const [inline]
```

Definition at line 159 of file [vec.h](#).

#### 5.2.4.11 capacity()

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline]
```

Definition at line 186 of file [vec.h](#).

#### 5.2.4.12 cbegin()

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin () const [inline]
```

Definition at line 164 of file [vec.h](#).

#### 5.2.4.13 cend()

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend () const [inline]
```

Definition at line 165 of file [vec.h](#).

#### 5.2.4.14 clear()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline]
```

Definition at line 197 of file [vec.h](#).

#### 5.2.4.15 crbegin()

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crbegin () const [inline]
```

Definition at line 170 of file [vec.h](#).

#### 5.2.4.16 crend()

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crend () const [inline]
```

Definition at line 171 of file [vec.h](#).

#### 5.2.4.17 data() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
T * Vector< T, Allocator >::data () [inline]
```

Definition at line 152 of file [vec.h](#).

#### 5.2.4.18 data() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::data () const [inline]
```

Definition at line 153 of file [vec.h](#).

#### 5.2.4.19 emplace()

```
template<class T, class Allocator = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Allocator >::emplace (
    iterator pos,
    Args &&... args) [inline]
```

Definition at line 228 of file [vec.h](#).

#### 5.2.4.20 emplace\_back()

```
template<class T, class Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Definition at line 268 of file [vec.h](#).

#### 5.2.4.21 empty()

```
template<class T, class Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () const [inline]
```

Definition at line 176 of file [vec.h](#).

**5.2.4.22 end()** [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end () [inline]
```

Definition at line 161 of file [vec.h](#).

**5.2.4.23 end()** [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end () const [inline]
```

Definition at line 162 of file [vec.h](#).

**5.2.4.24 erase()** [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    iterator first,
    iterator last) [inline]
```

Definition at line 250 of file [vec.h](#).

**5.2.4.25 erase()** [2/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    iterator pos) [inline]
```

Definition at line 241 of file [vec.h](#).

**5.2.4.26 front()** [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front () [inline]
```

Definition at line 146 of file [vec.h](#).

**5.2.4.27 front()** [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front () const [inline]
```

Definition at line 147 of file [vec.h](#).

#### 5.2.4.28 `get_allocator()`

```
template<class T, class Allocator = std::allocator<T>>
std::allocator< T > Vector< T, Allocator >::get_allocator () const [inline]
```

Definition at line 126 of file [vec.h](#).

#### 5.2.4.29 `insert()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    iterator pos,
    const T & value) [inline]
```

Definition at line 199 of file [vec.h](#).

#### 5.2.4.30 `insert()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    iterator pos,
    size_type count,
    const T & value) [inline]
```

Definition at line 212 of file [vec.h](#).

#### 5.2.4.31 `max_size()`

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline]
```

Definition at line 179 of file [vec.h](#).

#### 5.2.4.32 `operator=()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Definition at line 73 of file [vec.h](#).

#### 5.2.4.33 `operator=()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Definition at line 84 of file [vec.h](#).

**5.2.4.34 operator[]() [1/2]**

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type i) [inline]
```

Definition at line 143 of file [vec.h](#).

**5.2.4.35 operator[]() [2/2]**

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type i) const [inline]
```

Definition at line 144 of file [vec.h](#).

**5.2.4.36 pop\_back()**

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

Definition at line 283 of file [vec.h](#).

**5.2.4.37 push\_back()**

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & value) [inline]
```

Definition at line 260 of file [vec.h](#).

**5.2.4.38 rbegin()**

```
template<class T, class Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin () [inline]
```

Definition at line 167 of file [vec.h](#).

**5.2.4.39 rend()**

```
template<class T, class Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend () [inline]
```

Definition at line 168 of file [vec.h](#).

#### 5.2.4.40 reserve()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type new_cap) [inline]
```

Definition at line 181 of file [vec.h](#).

#### 5.2.4.41 resize()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type new_size) [inline]
```

Definition at line 289 of file [vec.h](#).

#### 5.2.4.42 shrink\_to\_fit()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Definition at line 188 of file [vec.h](#).

#### 5.2.4.43 size()

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline]
```

Definition at line 178 of file [vec.h](#).

#### 5.2.4.44 swap()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Definition at line 299 of file [vec.h](#).

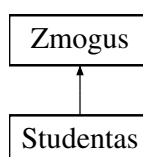
The documentation for this class was generated from the following file:

- [vec.h](#)

## 5.3 Zmogus Class Reference

```
#include <header.h>
```

Inheritance diagram for Zmogus:





## Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual std::string [vardas](#) () const
- virtual std::string [pavarde](#) () const
- virtual void [setVardas](#) (std::string &v)
- virtual void [setPavarde](#) (std::string &p)
- virtual void [spausdinti](#) () const =0
- virtual [~Zmogus](#) ()

## Protected Attributes

- std::string [vardas\\_](#)
- std::string [pavarde\\_](#)

### 5.3.1 Detailed Description

Definition at line 12 of file [header.h](#).

### 5.3.2 Constructor & Destructor Documentation

#### 5.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Definition at line 17 of file [header.h](#).

#### 5.3.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const std::string & vardas,  
    const std::string & pavarde) [inline]
```

Definition at line 18 of file [header.h](#).

#### 5.3.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Definition at line 28 of file [header.h](#).

### 5.3.3 Member Function Documentation

#### 5.3.3.1 pavarde()

```
virtual std::string Zmogus::pavarde () const [inline], [virtual]
```

Definition at line 21 of file [header.h](#).

#### 5.3.3.2 setPavarde()

```
virtual void Zmogus::setPavarde (  
    std::string & p) [inline], [virtual]
```

Definition at line 24 of file [header.h](#).

#### 5.3.3.3 setVardas()

```
virtual void Zmogus::setVardas (  
    std::string & v) [inline], [virtual]
```

Definition at line 23 of file [header.h](#).

#### 5.3.3.4 spausdinti()

```
virtual void Zmogus::spausdinti () const [pure virtual]
```

Implemented in [Studentas](#).

#### 5.3.3.5 vardas()

```
virtual std::string Zmogus::vardas () const [inline], [virtual]
```

Definition at line 20 of file [header.h](#).

### 5.3.4 Member Data Documentation

#### 5.3.4.1 pavarde\_

```
std::string Zmogus::pavarde_ [protected]
```

Definition at line 15 of file [header.h](#).

#### 5.3.4.2 vardas\_

```
std::string Zmogus::vardas_ [protected]
```

Definition at line 14 of file [header.h](#).

The documentation for this class was generated from the following file:

- [header.h](#)

# Chapter 6

## File Documentation

### 6.1 funkcijos.cpp File Reference

```
#include "funkcijos.h"
#include "vec.h"
#include <vector>
#include <iomanip>
#include <iostream>
#include <algorithm>
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <windows.h>
#include <sstream>
#include <list>
#include <deque>
```

#### Typedefs

- using laik = std::chrono::high\_resolution\_clock
- typedef std::uniform\_int\_distribution< int > int\_dis

#### Functions

- Studentas gen\_vrd ()
- int gen\_pazym ()
- void skaiciai (long &n, long &m)
- template<typename Konteineris>  
void skt\_visi (Konteineris &A, const string &failopav)
- void skt (Vector< Studentas > &A, string failopav)
- void skt (std::list< Studentas > &A, string failopav)
- void skt (std::deque< Studentas > &A, string failopav)
- void isved (Vector< Studentas > &A)
- void failu\_kurimas ()

- `template<typename Konteineris>`  
`void isvedimas_faila_visi (Konteineris &A, string pav)`
- `void isvedimas_faila (Vector< Studentas > &A, string pav)`
- `void isvedimas_faila (std::list< Studentas > &A, string pav)`
- `void isvedimas_faila (std::deque< Studentas > &A, string pav)`
- `int pasirink ()`
- `auto rikiav_visi (int rik)`
- `void rikiav (Vector< Studentas > &A, int rik)`
- `void rikiav (std::list< Studentas > &A, int rik)`
- `void rikiav (std::deque< Studentas > &A, int rik)`
- `template<typename Konteineris>`  
`void rusiavimas_visi (Konteineris &A, Konteineris &vargsai, Konteineris &kietekai)`
- `void rusiavimas (Vector< Studentas > &A, Vector< Studentas > &vargsai, Vector< Studentas > &kietekai)`
- `void rusiavimas (std::list< Studentas > &A, std::list< Studentas > &vargsai, std::list< Studentas > &kietekai)`
- `void rusiavimas (std::deque< Studentas > &A, std::deque< Studentas > &vargsai, std::deque< Studentas > &kietekai)`
- `template<typename Konteineris>`  
`void strat2_visi (Konteineris &A, Konteineris &vargsai)`
- `void strat2 (Vector< Studentas > &A, Vector< Studentas > &vargsai)`
- `void strat2 (std::list< Studentas > &A, std::list< Studentas > &vargsai)`
- `void strat2 (std::deque< Studentas > &A, std::deque< Studentas > &vargsai)`
- `template<typename Konteineris>`  
`void strat3_visi (Konteineris &A, Konteineris &vargsai)`
- `void strat3 (Vector< Studentas > &A, Vector< Studentas > &vargsai)`
- `void strat3 (std::list< Studentas > &A, std::list< Studentas > &vargsai)`
- `void strat3 (std::deque< Studentas > &A, std::deque< Studentas > &vargsai)`
- `void tyrimas1 ()`
- `void tyrimas2 (Vector< Studentas > &A)`
- `template<typename Konteineris>`  
`void tyrimas2_visi (Konteineris &A)`
- `void tyrimas2_ve (Vector< Studentas > &A)`
- `void tyrimas2_li (std::list< Studentas > &A)`
- `void tyrimas2_de (std::deque< Studentas > &A)`
- `void check (bool salyga, const string pavad)`
- `void testavimas ()`
- `void laiko_test ()`
- `void atminties_testas ()`

## 6.1.1 Typedef Documentation

### 6.1.1.1 int\_dis

```
typedef std::uniform_int_distribution<int> int_dis
```

Definition at line 29 of file [funkcijos.cpp](#).

### 6.1.1.2 laik

```
using laik = std::chrono::high_resolution_clock
```

Definition at line 28 of file [funkcijos.cpp](#).

## 6.1.2 Function Documentation

### 6.1.2.1 atminties\_testas()

```
void atminties_testas ()
```

Definition at line 530 of file [funkcijos.cpp](#).

### 6.1.2.2 check()

```
void check (  
    bool salyga,  
    const string pavad)
```

Definition at line 438 of file [funkcijos.cpp](#).

### 6.1.2.3 failu\_kurimas()

```
void failu_kurimas ()
```

Definition at line 149 of file [funkcijos.cpp](#).

### 6.1.2.4 gen\_pazym()

```
int gen_pazym ()
```

Definition at line 52 of file [funkcijos.cpp](#).

### 6.1.2.5 gen\_vrd()

```
Studentas gen_vrd ()
```

Definition at line 31 of file [funkcijos.cpp](#).

### 6.1.2.6 isved()

```
void isved (  
    Vector< Studentas > & A)
```

Definition at line 116 of file [funkcijos.cpp](#).

### 6.1.2.7 isvedimas\_faila() [1/3]

```
void isvedimas_faila (  
    std::deque< Studentas > & A,  
    string pav)
```

Definition at line 190 of file [funkcijos.cpp](#).

#### 6.1.2.8 isvedimas\_faila() [2/3]

```
void isvedimas_faila (
    std::list< Studentas > & A,
    string pav)
```

Definition at line 189 of file [funkcijos.cpp](#).

#### 6.1.2.9 isvedimas\_faila() [3/3]

```
void isvedimas_faila (
    Vector< Studentas > & A,
    string pav)
```

Definition at line 188 of file [funkcijos.cpp](#).

#### 6.1.2.10 isvedimas\_faila\_visi()

```
template<typename Konteineris>
void isvedimas_faila_visi (
    Konteineris & A,
    string pav)
```

Definition at line 179 of file [funkcijos.cpp](#).

#### 6.1.2.11 laiko\_test()

```
void laiko_test ()
```

Definition at line 506 of file [funkcijos.cpp](#).

#### 6.1.2.12 pasirink()

```
int pasirink ()
```

Definition at line 192 of file [funkcijos.cpp](#).

#### 6.1.2.13 rikiav() [1/3]

```
void rikiav (
    std::deque< Studentas > & A,
    int rik)
```

Definition at line 247 of file [funkcijos.cpp](#).

**6.1.2.14 rikiav()** [2/3]

```
void rikiav (
    std::list< Studentas > & A,
    int rik)
```

Definition at line 246 of file [funkcijos.cpp](#).

**6.1.2.15 rikiav()** [3/3]

```
void rikiav (
    Vector< Studentas > & A,
    int rik)
```

Definition at line 245 of file [funkcijos.cpp](#).

**6.1.2.16 rikiav\_visi()**

```
auto rikiav_visi (
    int rik)
```

Definition at line 220 of file [funkcijos.cpp](#).

**6.1.2.17 rusiavimas()** [1/3]

```
void rusiavimas (
    std::deque< Studentas > & A,
    std::deque< Studentas > & vargsai,
    std::deque< Studentas > & kietekai)
```

Definition at line 262 of file [funkcijos.cpp](#).

**6.1.2.18 rusiavimas()** [2/3]

```
void rusiavimas (
    std::list< Studentas > & A,
    std::list< Studentas > & vargsai,
    std::list< Studentas > & kietekai)
```

Definition at line 261 of file [funkcijos.cpp](#).

**6.1.2.19 rusiavimas()** [3/3]

```
void rusiavimas (
    Vector< Studentas > & A,
    Vector< Studentas > & vargsai,
    Vector< Studentas > & kietekai)
```

Definition at line 260 of file [funkcijos.cpp](#).

#### 6.1.2.20 rusiavimas\_visi()

```
template<typename Konteineris>
void rusiavimas_visi (
    Konteineris & A,
    Konteineris & vargsai,
    Konteineris & kietekai)
```

Definition at line 252 of file [funkcijos.cpp](#).

#### 6.1.2.21 skaiciai()

```
void skaiciai (
    long & n,
    long & m)
```

Definition at line 59 of file [funkcijos.cpp](#).

#### 6.1.2.22 skt() [1/3]

```
void skt (
    std::deque< Studentas > & A,
    string failopav)
```

Definition at line 114 of file [funkcijos.cpp](#).

#### 6.1.2.23 skt() [2/3]

```
void skt (
    std::list< Studentas > & A,
    string failopav)
```

Definition at line 113 of file [funkcijos.cpp](#).

#### 6.1.2.24 skt() [3/3]

```
void skt (
    Vector< Studentas > & A,
    string failopav)
```

Definition at line 112 of file [funkcijos.cpp](#).

#### 6.1.2.25 skt\_visi()

```
template<typename Konteineris>
void skt_visi (
    Konteineris & A,
    const string & failopav)
```

Definition at line 91 of file [funkcijos.cpp](#).



**6.1.2.26 strat2()** [1/3]

```
void strat2 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 294 of file [funkcijos.cpp](#).

**6.1.2.27 strat2()** [2/3]

```
void strat2 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 293 of file [funkcijos.cpp](#).

**6.1.2.28 strat2()** [3/3]

```
void strat2 (  
    Vector< Studentas > & A,  
    Vector< Studentas > & vargsai)
```

Definition at line 292 of file [funkcijos.cpp](#).

**6.1.2.29 strat2\_visi()**

```
template<typename Konteineris>  
void strat2_visi (  
    Konteineris & A,  
    Konteineris & vargsai)
```

Definition at line 266 of file [funkcijos.cpp](#).

**6.1.2.30 strat3()** [1/3]

```
void strat3 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 320 of file [funkcijos.cpp](#).

**6.1.2.31 strat3()** [2/3]

```
void strat3 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 319 of file [funkcijos.cpp](#).

#### 6.1.2.32 strat3() [3/3]

```
void strat3 (  
    Vector< Studentas > & A,  
    Vector< Studentas > & vargsai)
```

Definition at line 318 of file [funkcijos.cpp](#).

#### 6.1.2.33 strat3\_visi()

```
template<typename Konteineris>  
void strat3_visi (  
    Konteineris & A,  
    Konteineris & vargsai)
```

Definition at line 298 of file [funkcijos.cpp](#).

#### 6.1.2.34 testavimas()

```
void testavimas ()
```

Definition at line 443 of file [funkcijos.cpp](#).

#### 6.1.2.35 tyrimas1()

```
void tyrimas1 ()
```

Definition at line 322 of file [funkcijos.cpp](#).

#### 6.1.2.36 tyrimas2()

```
void tyrimas2 (  
    Vector< Studentas > & A)
```

Definition at line 330 of file [funkcijos.cpp](#).

#### 6.1.2.37 tyrimas2\_de()

```
void tyrimas2_de (  
    std::deque< Studentas > & A)
```

Definition at line 434 of file [funkcijos.cpp](#).

#### 6.1.2.38 tyrimas2\_li()

```
void tyrimas2_li (  
    std::list< Studentas > & A)
```

Definition at line 433 of file [funkcijos.cpp](#).

## 6.1.2.39 tyrimas2\_ve()

```
void tyrimas2_ve (
    Vector< Studentas > & A)
```

Definition at line 432 of file funkcijos.cpp.

## 6.1.2.40 tyrimas2\_visi()

```
template<typename Konteineris>
void tyrimas2_visi (
    Konteineris & A)
```

Definition at line 375 of file funkcijos.cpp.

## 6.2 funkcijos.cpp

[Go to the documentation of this file.](#)

```
00001 #include "funkcijos.h"
00002 #include "vec.h"
00003 #include <vector>
00004 #include <iomanip>
00005 #include <iostream>
00006 #include <algorithm>
00007 #include <random>
00008 #include <chrono>
00009 #include <limits>
00010 #include <fstream>
00011 #include <windows.h>
00012 #include <sstream>
00013 #include <list>
00014 #include <deque>
00015
00016
00017 using std::cout;
00018 using std::cin;
00019 using std::endl;
00020 using std::setw;
00021 using std::left;
00022 using std::mt19937;
00023 using std::setprecision;
00024 using std::fixed;
00025 using std::string;
00026 using std::getline;
00027
00028 using laik = std::chrono::high_resolution_clock;
00029 typedef std::uniform_int_distribution<int> int_dis;
00030
00031 Studentas gen_vrd() {
00032     Studentas A;
00033     mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00034     int_dis paskirst(0,9);
00035     std::string v[10] = {"Jonas", "Paulius", "Saulius", "Matas", "Egle", "Ruta", "Rima", "Lukas",
00036 "Juste", "Laura"};
00037     std::string p_v[10] = {"Jonaitis", "Saulenas", "Pavardaitis", "Vardaitis", "Rimauskas",
00038 "Paulauskas", "Mataitis", "Saulaitis", "Jonauskas", "Jokubaitis"};
00039     std::string p_m[10] = {"Eglaite", "Rutaite", "Rimaite", "Justiene", "Lauraite", "Joniene",
00040 "Jonaite", "Mataite", "Paulaitiene", "Pavardiene"};
00041     A.setVardas(v[paskirst(rnd)]);
00042     switch (*A.vardas().rbegin())
00043     {
00044     case 's' :
00045         A.setPavarde(p_v[paskirst(rnd)]);
00046         break;
00047     default:
00048         A.setPavarde(p_m[paskirst(rnd)]);
00049         break;
00050     };
00051     return A;
00052 }
```

```

00051
00052 int gen_pazym() {
00053     mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00054     int_dis paskirst(1,10);
00055
00056     return paskirst(rnd);
00057 };
00058
00059 void skaiciai(long &n, long &m){
00060     cout<<"Įvesk kiek pažymių gavo už namų darbus"<<endl;
00061     while(true){
00062         cin>>n;
00063         if(cin.fail() || n<=0){
00064             cin.clear();
00065             cin.ignore(10000, '\n');
00066             cout<<"Įvesti galima tik sveikuosius teigiamus skaičius"<<endl;
00067         }
00068         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00069             cin.ignore(10000, '\n');
00070             cout<<"Įvesti galima tik sveikuosius skaičius"<<endl;
00071         }
00072         else{break;}
00073     }
00074     cout<<"Įvesk studentų skaičių"<<endl;
00075     while(true){
00076         cin>>m;
00077         if(cin.fail() || m<=0){
00078             cin.clear();
00079             cin.ignore(10000, '\n');
00080             cout<<"Įvesti galima tik sveikuosius teigiamus skaičius"<<endl;
00081         }
00082         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00083             cin.ignore(10000, '\n');
00084             cout<<"Įvesti galima tik sveikuosius skaičius"<<endl;
00085         }
00086         else{break;}
00087     }
00088 }
00089
00090 template <typename Konteineris>
00091 void skt_visi(Konteineris& A, const string& failopav){
00092     //std::stringstream buf;
00093
00094     std::ifstream f(failopav);
00095
00096     string eilut;
00097     if(!f.is_open()){throw std::runtime_error("Nepavyko atidaryti failo");}
00098 }
00099
00100     std::string antr;
00101     getline(f, antr); // antraste
00102
00103     Studentas tmp;
00104     while (f>tmp)
00105     {
00106         A.push_back(tmp);
00107     }
00108
00109     f.close();
00110 }
00111
00112 void skt(Vector<Studentas>& A, string failopav){skt_visi(A,failopav);}
00113 void skt(std::list<Studentas>& A, string failopav){skt_visi(A,failopav);}
00114 void skt(std::deque<Studentas>& A, string failopav){skt_visi(A,failopav);}
00115
00116 void isved(Vector<Studentas>& A){
00117     int is;
00118     cout<<"1 - Išvesti su vidurkiu, 2 - Išvesti su mediana"<<endl;
00119     while(true){
00120         cin>>is;
00121         if(cin.fail() || is<1 || is>2){
00122             cin.clear();
00123             cin.ignore(10000, '\n');
00124             cout<<"Įvesti galima tik skaičius 1 arba 2"<<endl;
00125         }
00126         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00127             cin.ignore(10000, '\n');
00128             cout<<"Įvesti galima tik sveikuosius skaičius"<<endl;
00129         }
00130         else{break;}
00131     }
00132
00133     if(is==1){
00134         cout<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<"Galutinis (Vid.)"<<endl;
00135         cout<<"-----"<<endl;
00136         for(const auto& s: A){
00137

```

```

        cout<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<endl;
00138    }
00139    }
00140    if(is==2){
00141        cout<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<"Galutinis (Med.)"<<endl;
00142        cout<<"-----"<<endl;
00143        for(const auto& s: A){
00144            cout<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<fixed<<setprecision(2)<<s.galutBalas()<<endl;
00145        }
00146    }
00147 }
00148
00149 void failu_kurimas(){
00150     Vector<long> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00151
00152     for(long y: kiekiai){
00153         auto startas = std::chrono::high_resolution_clock::now();
00154         string pavad = "failas"+std::to_string(y)+".txt";
00155         std::ofstream f(pavad);
00156         f<<left<<setw(25)<<"Vardas"<<setw(25)<<"Pavardė";
00157         for(int i=1; i<=15; i++){
00158             f<<setw(10)<<("ND"+std::to_string(i));
00159         }
00160         f<<"Egz."<<endl; // pirma eil
00161
00162         mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00163         int_dis paskirst(1,10);
00164
00165         for(long i=1; i<=y; i++){
00166             f<<left<<setw(25)<<("VardasN"+std::to_string(i))<<setw(25)<<("PavardėN"+std::to_string(i));
00167             for(int j=0; j<15; j++){
00168                 f<<setw(10)<<paskirst(rnd);
00169             }
00170             f<<paskirst(rnd)<<endl;
00171         }
00172         f.close();
00173         std::chrono::duration<double> diff=laik::now()-startas;
00174         cout<<pavad<<" failo kūrimas užtruko "<<diff.count()<<" s"<<endl;
00175     }
00176 }
00177
00178 template <typename Konteineris>
00179 void isvedimas_faila_visi(Konteineris& A, string pav){
00180     std::ofstream r(pav);
00181     r<<left<<setw(20)<<"Pavardė"<<setw(20)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis (Med.)"<<"\n";
00182     r<<"-----"<<"\n";
00183     for(const auto& s : A){
00184         r<<left<<setw(20)<<s.pavarde()<<setw(20)<<s.vardas()<<setw(17)<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<<setprecision(2)<<endl;
00185     }
00186     r.close();
00187 }
00188 void isvedimas_faila(Vector<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00189 void isvedimas_faila(std::list<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00190 void isvedimas_faila(std::deque<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00191
00192 int pasirink(){
00193     int rik;
00194     cout<<"1 - Rikiuosti pagal vardą mažėjančiai\n"
00195         <<"2 - pagal vardą didėjančiai\n"
00196         <<"3 - pagal pavardę mažėjančiai\n"
00197         <<"4 - pagal pavardę didėjančiai\n"
00198         <<"5 - pagal vidurkį mažėjančiai\n"
00199         <<"6 - pagal vidurkį didėjančiai\n"
00200         <<"7 - pagal medianą mažėjančiai\n"
00201         <<"8 - pagal medianą didėjančiai"<<endl;
00202     while(true){
00203         cin>>rik;
00204         if(cin.fail() || rik<1 || rik>8){
00205             cin.clear();
00206             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00207             cout<<"Ivesti galima tik 1, 2, 3, 4, 5, 6, 7 arba 8"<<endl;
00208         }
00209         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00210             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00211             cout<<"Ivesti galima tik 1, 2, 3, 4, 5, 6, 7 arba 8"<<endl;
00212         }
00213         else{
00214             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00215             break;
00216         }
00217         return rik;
00218     }
00219 }
00220 auto rikiav_visi(int rik){
00221     return [rik](const Studentas& A, const Studentas& B){

```

```

00222         switch (rik)
00223         {
00224             case 1:
00225                 return A.vardas() > B.vardas();
00226             case 2:
00227                 return A.vardas() < B.vardas();
00228             case 3:
00229                 return A.pavarde() > B.pavarde() ;
00230             case 4:
00231                 return A.pavarde() < B.pavarde() ;
00232             case 5:
00233                 return A.galutBalas(vidurkis) > B.galutBalas(vidurkis);
00234             case 6:
00235                 return A.galutBalas(vidurkis) < B.galutBalas(vidurkis);
00236             case 7:
00237                 return A.galutBalas() > B.galutBalas();
00238             case 8:
00239                 return A.galutBalas() < B.galutBalas();
00240             default: return false;
00241         }
00242     };
00243 };
00244 }
00245 void rikiav(Vector<Studentas>& A, int rik){std::sort(A.begin(), A.end(), rikiav_visi(rik));}
00246 void rikiav(std::list<Studentas>& A, int rik){A.sort(rikiav_visi(rik));}
00247 void rikiav(std::deque<Studentas>& A, int rik){std::sort(A.begin(), A.end(), rikiav_visi(rik));}
00248
00249
00250 // 1 STRATEGIJA
00251 template <typename Konteineris>
00252 void rusiavimas_visi(Konteineris& A, Konteineris& vargsai, Konteineris& kietekai){
00253     for(const auto& s: A){
00254         if(s.galutBalas(vidurkis)<5.0){
00255             vargsai.push_back(s);
00256         }
00257         else{kietekai.push_back(s);}
00258     }
00259 }
00260 void rusiavimas(Vector<Studentas>& A, Vector<Studentas>& vargsai, Vector<Studentas>&
kietekai){rusiavimas_visi(A,vargsai,kietekai);}
00261 void rusiavimas(std::list<Studentas>& A, std::list<Studentas>& vargsai, std::list<Studentas>&
kietekai){rusiavimas_visi(A,vargsai,kietekai);}
00262 void rusiavimas(std::deque<Studentas>& A, std::deque<Studentas>& vargsai, std::deque<Studentas>&
kietekai){rusiavimas_visi(A,vargsai,kietekai);}
00263
00264 // 2 STRATEGIJA
00265 template <typename Konteineris>
00266 void strat2_visi(Konteineris& A, Konteineris& vargsai){
00267     if constexpr(std::is_same_v<Konteineris, Vector<Studentas> ||
std::is_same_v<Konteineris, std::deque<Studentas>){
00268         size_t i = 0;
00269         while(i < A.size()){
00270             if(A[i].galutBalas() < 5.0){
00271                 vargsai.push_back(A[i]);
00272                 A[i] = A.back();
00273                 A.pop_back();
00274             }
00275             else{++i;}
00276         }
00277     }
00278     else{
00279         vargsai.clear();
00280         for (auto it = A.begin(); it != A.end(); ) {
00281             if (it->galutBalas() < 5.0) {
00282                 vargsai.push_back(*it);
00283                 it = A.erase(it);
00284             } else {
00285                 ++it;
00286             }
00287         }
00288     }
00289 }
00290 }
00291
00292 void strat2(Vector<Studentas>& A, Vector<Studentas>& vargsai){strat2_visi(A,vargsai);}
00293 void strat2(std::list<Studentas>& A, std::list<Studentas>& vargsai){strat2_visi(A,vargsai);}
00294 void strat2(std::deque<Studentas>& A, std::deque<Studentas>& vargsai){strat2_visi(A,vargsai);}
00295
00296 // 3 STRATEGIJA
00297 template <typename Konteineris>
00298 void strat3_visi(Konteineris& A, Konteineris& vargsai){
00299     vargsai.clear();
00300     if constexpr(std::is_same_v<Konteineris, std::list<Studentas>){
00301         A.remove_if([&](const Studentas& s){
00302             if(s.galutBalas() < 5.0){
00303                 vargsai.push_back(s);
00304             }
00305         });
00306     }
00307     else{
00308         return true;
00309     }
00310 }

```

```

00305         }
00306         return false;
00307     });
00308 }
00309
00310 else {auto it = std::partition(A.begin(), A.end(), [](const Studentas& s){
00311     return s.galutBalas()>=5.0;
00312 });
00313 for(auto i = it; i != A.end(); ++i){
00314     vargsai.push_back(std::move(*i));
00315     A.erase(it, A.end());
00316 }
00317 }
00318 void strat3(Vector<Studentas>& A, Vector<Studentas>& vargsai){strat3_visi(A,vargsai);}
00319 void strat3(std::list<Studentas>& A, std::list<Studentas>& vargsai){strat3_visi(A,vargsai);}
00320 void strat3(std::deque<Studentas>& A, std::deque<Studentas>& vargsai){strat3_visi(A,vargsai);}
00321
00322 void tyrimas1(){
00323     auto startas = std::chrono::high_resolution_clock::now();
00324     failu_kurimas();
00325     std::chrono::duration<double> diff=laik::now()-startas;
00326     cout<<"Visų failų kūrimas užtruko "<<diff.count()<<" s"<<endl;
00327 }
00328
00329
00330 void tyrimas2(Vector<Studentas>& A){
00331     Vector<long> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00332     int rik = pasirink();
00333     auto startas = std::chrono::high_resolution_clock::now();
00334
00335     for(long y: kiekiai){
00336         A.clear();
00337         A.reserve(y);
00338         string pavad = "failas"+std::to_string(y)+".txt";
00339
00340         cout<<y<<" failas : "<<endl;
00341         auto til= std::chrono::high_resolution_clock::now(); // nuskaitymas
00342         try{
00343             skt(A, pavad);
00344         }
00345         catch(const std::exception& e){
00346             std::cerr<<"Klaida : " <<e.what()<<endl;
00347             return void();
00348         }
00349
00350         std::chrono::duration<double> diff1=laik::now()-til;
00351         cout<<"Failo nuskaitymas užtruko "<<diff1.count()<<" s\n";
00352
00353         auto ti3= std::chrono::high_resolution_clock::now(); // rusiavimas
00354         Vector<Studentas> vargsai, kietekai;
00355         rusiavimas(A,vargsai, kietekai);
00356         rikiav(vargsai,rik);
00357         rikiav(kietekai,rik);
00358         std::chrono::duration<double> diff3=laik::now()-ti3;
00359         cout<<"Studentų rūšiavimas užtruko "<<diff3.count()<<" s\n";
00360
00361         auto ti4= std::chrono::high_resolution_clock::now(); // rasymas i failus
00362         isvedimas_faila(vargsai, "vargsai.txt");
00363         isvedimas_faila(kietekai, "kietekai.txt");
00364         std::chrono::duration<double> diff4=laik::now()-ti4;
00365         cout<<"Rašymas į failus užtruko "<<diff4.count()<<" s\n";
00366         cout<<"\n";
00367     }
00368
00369     std::chrono::duration<double> diff=laik::now()-startas;
00370     cout<<"Visos programos veikimas užtruko "<<diff.count()<<" s\n";
00371
00372
00373 }
00374 template <typename Kontaineris>
00375 void tyrimas2_visi(Kontaineris& A){
00376     Vector<long> kiekiai = {100000, 1000000, 10000000};
00377     int rik = pasirink();
00378     auto startas = std::chrono::high_resolution_clock::now();
00379
00380     for(long y: kiekiai){
00381         A.clear();
00382         string pavad = "failas"+std::to_string(y)+".txt";
00383
00384         cout<<y<<" failas : "<<endl;
00385         // nuskaitymas
00386         auto til= std::chrono::high_resolution_clock::now();
00387         try{
00388             skt(A, pavad);
00389         }
00390         catch(const std::exception& e){
00391             std::cerr<<"Klaida : " <<e.what()<<endl;

```

```

00392         return void();
00393     }
00394     std::chrono::duration<double> diff1=laik::now()-ti1;
00395     cout<<"Failo nuskaitymas užtruko "<<diff1.count()<<" s\n";
00396
00397     // rusiavimas
00398     // 1 STRATEGIJA
00399     Konteineris vargsai, kietekai, cop(A), c1(A), c2(A);
00400     vargsai.clear();
00401     auto ti3= std::chrono::high_resolution_clock::now();
00402     rusiavimas(cop,vargsai, kietekai);
00403     std::chrono::duration<double> diff3=laik::now()-ti3;
00404     cout<<"1 Strategija studentų rūšiavimas užtruko "<<diff3.count()<<" s\n";
00405
00406     // 2 STRATEGIJA
00407     vargsai.clear();
00408     auto st2= std::chrono::high_resolution_clock::now();
00409     strat2(c1,vargsai);
00410     std::chrono::duration<double> stdif2=laik::now()-st2;
00411     cout<<"2 Strategija studentų rūšiavimas užtruko "<<stdif2.count()<<" s\n";
00412
00413     // 3 STRATEGIJA
00414     vargsai.clear();
00415     auto st3= std::chrono::high_resolution_clock::now();
00416     strat3(c2,vargsai);
00417     std::chrono::duration<double> stdif3=laik::now()-st3;
00418     cout<<"3 Strategija studentų rūšiavimas užtruko "<<stdif3.count()<<" s\n";
00419
00420     // rikiavimas
00421     auto ti4= std::chrono::high_resolution_clock::now();
00422     rikiav(vargsai,rik);
00423     rikiav(c2,rik);
00424     std::chrono::duration<double> diff4=laik::now()-ti4;
00425     cout<<"Studentų rikiavimas užtruko "<<diff4.count()<<" s\n";
00426
00427     isvedimas_faila(vargsai, "vargsai.txt");
00428     isvedimas_faila(c2,"kietekai.txt");
00429
00430 }
00431 }
00432 void tyrimas2_ve(Vector<Studentas>& A){tyrimas2_visi(A);}
00433 void tyrimas2_li(std::list<Studentas>& A){tyrimas2_visi(A);}
00434 void tyrimas2_de(std::deque<Studentas>& A){tyrimas2_visi(A);}
00435
00436
00437 // rule of five testavimas
00438 void check(bool salyga, const string pavad){
00439     if(salyga){cout<<pavad<<" - veikia teisingai\n";}
00440     else{cout<<pavad<<" - Neveikia teisingai\n";}
00441 }
00442
00443 void testavimas(){
00444     Vector<double> nd = {8, 9, 5};
00445     Studentas pradinis("Jonas", "Jonaitis", 10, nd);
00446
00447     cout<<pradinis<<"\n";
00448
00449     Studentas kopija(pradinis);
00450     cout<<kopija<<"\n";
00451
00452     check(kopija.vardas()==pradinis.vardas(), "Kopija V"); // copy constr
00453     check(kopija.pavarde()==pradinis.pavarde(), "Kopija P"); // copy constr
00454     //check(kopija.nd()==pradinis.nd(), "Kopija ND"); // copy constr
00455     check(kopija.egzaminas()==pradinis.egzaminas(), "Kopija E"); // copy constr
00456
00457     cout<<"-----\n";
00458
00459
00460     Studentas move1(std::move(pradinis)); // move konstr
00461     //cout<<"move konstr - "<<move1<<"\n";
00462     check(move1.vardas()=="Jonas", "Move V");
00463     check(move1.pavarde()=="Jonaitis", "Move P");
00464     //check(move1.nd()== Vector<double>({8, 9, 5}), "Move ND");
00465     check(move1.egzaminas()==10, "Move E");
00466
00467     check(pradinis.vardas().empty(), "Move isvalymas V");
00468     check(pradinis.pavarde().empty(), "Move isvalymas P");
00469     check(pradinis.nd().empty(), "Move isvalymas ND");
00470
00471     cout<<"-----\n";
00472     Studentas org("Jonas", "Jonaitis", 10, nd);
00473     Studentas prisk; // priskyrimas
00474     prisk = org;
00475     check(prisk.vardas()==org.vardas(), "Priskyrimas V");
00476     check(prisk.pavarde()==org.pavarde(), "Priskyrimas P");
00477     //check(prisk.nd()==org.nd(), "Priskyrimas ND");
00478     check(prisk.egzaminas()==org.egzaminas(), "Priskyrimas E");

```



```

00479
00480     cout<<"-----\n";
00481
00482     Studentas move2; // priskyrimo move
00483     move2 = std::move(kopija);
00484     check(move2.vardas() == "Jonas", "Priskyrimo V move");
00485     check(move2.pavarde() == "Jonaitis", "Priskyrimo P move");
00486     // check(move2.nd() == Vector<double>({8, 9, 5}), "Priskyrimo ND move");
00487     check(move2.egzaminas() == 10, "Priskyrimo E move");
00488
00489     check(kopija.vardas().empty(), "Priskyrimo move isvalymas V");
00490     check(kopija.pavarde().empty(), "Priskyrimo move isvalymas P");
00491     check(kopija.nd().empty(), "Priskyrimo move isvalymas ND");
00492
00493     cout<<"-----\n";
00494
00495     std::istringstream is("Paulius Lukaitis 6 7 8 8"); // iverstis
00496     Studentas skaitymas;
00497     is>>skaitymas;
00498     cout<<skaitymas<<"\n";
00499
00500     std::ostringstream o; // isvestis
00501     o<<skaitymas;
00502     string isv = o.str();
00503     cout<<isv<<"\n";
00504
00505 }
00506 void laiko_test(){
00507     std::vector<int> size{10000, 100000, 1000000, 10000000, 100000000};
00508     std::vector<int> v1;
00509     Vector<int> v2;
00510
00511     for(int sz : size){
00512         auto startas = std::chrono::high_resolution_clock::now();
00513         for(int i = 1; i <= sz; i++){
00514             v1.push_back(i);
00515         }
00516         auto pabaiga = std::chrono::high_resolution_clock::now();
00517         std::chrono::duration<double> laikas = pabaiga - startas;
00518         cout<<sz<<" Su std::vector uztruko - "«laikas.count()«" s\n";
00519
00520         auto startas2 = std::chrono::high_resolution_clock::now();
00521         for(int i = 1; i <= sz; i++){
00522             v2.push_back(i);
00523         }
00524         auto pabaiga2 = std::chrono::high_resolution_clock::now();
00525         std::chrono::duration<double> laikas2 = pabaiga2 - startas2;
00526         cout<<sz<<" Su Vector uztruko - "«laikas2.count()«" s\n";
00527     }
00528 }
00529
00530 void atminties_testas(){
00531     int kiekis = 100000000;
00532     {
00533         std::vector<int> v;
00534         int perskirstymai = 0;
00535
00536         for (int i = 0; i < kiekis; ++i) {
00537             if (v.size() == v.capacity())
00538                 ++perskirstymai;
00539             v.push_back(i);
00540         }
00541         cout<<"Perskirstymai std::vector- "«perskirstymai<<endl;
00542     }
00543     {
00544         Vector<int> v;
00545         int perskirstymai = 0;
00546
00547         for (int i = 0; i < kiekis; ++i) {
00548             if (v.size() == v.capacity())
00549                 ++perskirstymai;
00550             v.push_back(i);
00551         }
00552         cout<<"Perskirstymai Vector- "«perskirstymai<<endl;
00553     }
00554 }
00555 }

```

## 6.3 funkcijos.h File Reference

```

#include <vector>
#include <string>

```

```
#include <list>
#include <deque>
#include "header.h"
#include "vec.h"
```

## Functions

- [Studentas gen\\_vrd](#) ()
- int [gen\\_pazym](#) ()
- void [skaiciai](#) (long &n, long &m)
- int [pasirink](#) ()
- void [isved](#) (Vector< [Studentas](#) > &A)
- void [isvedimas\\_faila](#) (Vector< [Studentas](#) > &A, std::string pav)
- void [isvedimas\\_faila](#) (std::list< [Studentas](#) > &A, std::string pav)
- void [isvedimas\\_faila](#) (std::deque< [Studentas](#) > &A, std::string pav)
- void [failu\\_kurimas](#) ()
- void [skt](#) (Vector< [Studentas](#) > &A, std::string failopav)
- void [skt](#) (std::list< [Studentas](#) > &A, std::string failopav)
- void [skt](#) (std::deque< [Studentas](#) > &A, std::string failopav)
- void [rikiav](#) (Vector< [Studentas](#) > &A, int rik)
- void [rikiav](#) (std::list< [Studentas](#) > &A, int rik)
- void [rikiav](#) (std::deque< [Studentas](#) > &A, int rik)
- void [rusiavimas](#) (Vector< [Studentas](#) > &A, Vector< [Studentas](#) > &vargsai, Vector< [Studentas](#) > &kietekai)
- void [rusiavimas](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai, std::list< [Studentas](#) > &kietekai)
- void [rusiavimas](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai, std::deque< [Studentas](#) > &kietekai)
- void [strat2](#) (Vector< [Studentas](#) > &A, Vector< [Studentas](#) > &vargsai)
- void [strat2](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void [strat2](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- void [strat3](#) (Vector< [Studentas](#) > &A, Vector< [Studentas](#) > &vargsai)
- void [strat3](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void [strat3](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- void [tyrimas1](#) ()
- void [tyrimas2](#) (Vector< [Studentas](#) > &A)
- void [tyrimas2\\_ve](#) (Vector< [Studentas](#) > &A)
- void [tyrimas2\\_li](#) (std::list< [Studentas](#) > &A)
- void [tyrimas2\\_de](#) (std::deque< [Studentas](#) > &A)
- void [check](#) (bool salyga, const std::string pavad)
- void [testavimas](#) ()
- void [laiko\\_test](#) ()
- void [atminties\\_testas](#) ()

## 6.3.1 Function Documentation

### 6.3.1.1 atminties\_testas()

```
void atminties_testas ()
```

Definition at line 530 of file [funkcijos.cpp](#).

#### 6.3.1.2 check()

```
void check (
    bool salyga,
    const std::string pavad)
```

#### 6.3.1.3 failu\_kurimas()

```
void failu_kurimas ()
```

Definition at line 149 of file [funkcijos.cpp](#).

#### 6.3.1.4 gen\_pazym()

```
int gen_pazym ()
```

Definition at line 52 of file [funkcijos.cpp](#).

#### 6.3.1.5 gen\_vrd()

```
Studentas gen_vrd ()
```

Definition at line 31 of file [funkcijos.cpp](#).

#### 6.3.1.6 isved()

```
void isved (
    Vector< Studentas > & A)
```

Definition at line 116 of file [funkcijos.cpp](#).

#### 6.3.1.7 isvedimas\_faila() [1/3]

```
void isvedimas_faila (
    std::deque< Studentas > & A,
    std::string pav)
```

#### 6.3.1.8 isvedimas\_faila() [2/3]

```
void isvedimas_faila (
    std::list< Studentas > & A,
    std::string pav)
```

#### 6.3.1.9 isvedimas\_faila() [3/3]

```
void isvedimas_faila (  
    Vector< Studentas > & A,  
    std::string pav)
```

#### 6.3.1.10 laiko\_test()

```
void laiko_test ()
```

Definition at line 506 of file [funkcijos.cpp](#).

#### 6.3.1.11 pasirink()

```
int pasirink ()
```

Definition at line 192 of file [funkcijos.cpp](#).

#### 6.3.1.12 rikiav() [1/3]

```
void rikiav (  
    std::deque< Studentas > & A,  
    int rik)
```

Definition at line 247 of file [funkcijos.cpp](#).

#### 6.3.1.13 rikiav() [2/3]

```
void rikiav (  
    std::list< Studentas > & A,  
    int rik)
```

Definition at line 246 of file [funkcijos.cpp](#).

#### 6.3.1.14 rikiav() [3/3]

```
void rikiav (  
    Vector< Studentas > & A,  
    int rik)
```

Definition at line 245 of file [funkcijos.cpp](#).

#### 6.3.1.15 rusiavimas() [1/3]

```
void rusiavimas (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai,  
    std::deque< Studentas > & kietekai)
```

Definition at line 262 of file [funkcijos.cpp](#).

**6.3.1.16 rusiavimas()** [2/3]

```
void rusiavimas (
    std::list< Studentas > & A,
    std::list< Studentas > & vargsai,
    std::list< Studentas > & kietekai)
```

Definition at line 261 of file [funkcijos.cpp](#).

**6.3.1.17 rusiavimas()** [3/3]

```
void rusiavimas (
    Vector< Studentas > & A,
    Vector< Studentas > & vargsai,
    Vector< Studentas > & kietekai)
```

Definition at line 260 of file [funkcijos.cpp](#).

**6.3.1.18 skaiciai()**

```
void skaiciai (
    long & n,
    long & m)
```

Definition at line 59 of file [funkcijos.cpp](#).

**6.3.1.19 skt()** [1/3]

```
void skt (
    std::deque< Studentas > & A,
    std::string failopav)
```

**6.3.1.20 skt()** [2/3]

```
void skt (
    std::list< Studentas > & A,
    std::string failopav)
```

**6.3.1.21 skt()** [3/3]

```
void skt (
    Vector< Studentas > & A,
    std::string failopav)
```

**6.3.1.22 strat2()** [1/3]

```
void strat2 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 294 of file [funkcijos.cpp](#).

**6.3.1.23 strat2()** [2/3]

```
void strat2 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 293 of file [funkcijos.cpp](#).

**6.3.1.24 strat2()** [3/3]

```
void strat2 (  
    Vector< Studentas > & A,  
    Vector< Studentas > & vargsai)
```

Definition at line 292 of file [funkcijos.cpp](#).

**6.3.1.25 strat3()** [1/3]

```
void strat3 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 320 of file [funkcijos.cpp](#).

**6.3.1.26 strat3()** [2/3]

```
void strat3 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 319 of file [funkcijos.cpp](#).

**6.3.1.27 strat3()** [3/3]

```
void strat3 (  
    Vector< Studentas > & A,  
    Vector< Studentas > & vargsai)
```

Definition at line 318 of file [funkcijos.cpp](#).

#### 6.3.1.28 testavimas()

```
void testavimas ()
```

Definition at line 443 of file [funkcijos.cpp](#).

#### 6.3.1.29 tyrimas1()

```
void tyrimas1 ()
```

Definition at line 322 of file [funkcijos.cpp](#).

#### 6.3.1.30 tyrimas2()

```
void tyrimas2 (  
    Vector< Studentas > & A)
```

Definition at line 330 of file [funkcijos.cpp](#).

#### 6.3.1.31 tyrimas2\_de()

```
void tyrimas2_de (  
    std::deque< Studentas > & A)
```

Definition at line 434 of file [funkcijos.cpp](#).

#### 6.3.1.32 tyrimas2\_li()

```
void tyrimas2_li (  
    std::list< Studentas > & A)
```

Definition at line 433 of file [funkcijos.cpp](#).

#### 6.3.1.33 tyrimas2\_ve()

```
void tyrimas2_ve (  
    Vector< Studentas > & A)
```

Definition at line 432 of file [funkcijos.cpp](#).

## 6.4 funkcijos.h

[Go to the documentation of this file.](#)

```

00001 #ifndef funkcijos_H
00002 #define funkcijos_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <list>
00007 #include <deque>
00008 #include "header.h"
00009 #include "vec.h"
00010
00011
00012 // generavimas
00013 Studentas gen_vrd();
00014 int gen_pazym();
00015
00016 void skaiciai(long &n, long &m);
00017 int pasirink();
00018
00019 // isvedimai
00020 void isved(Vector<Studentas>& A);
00021 void isvedimas_faila(Vector<Studentas>& A, std::string pav);
00022 void isvedimas_faila(std::list<Studentas>& A, std::string pav);
00023 void isvedimas_faila(std::deque<Studentas>& A, std::string pav);
00024
00025 void failu_kurimas();
00026
00027 // skaitymas
00028 void skt(Vector<Studentas>& A, std::string failopav);
00029 void skt(std::list<Studentas>& A, std::string failopav);
00030 void skt(std::deque<Studentas>& A, std::string failopav);
00031
00032 // rikiavimas
00033 void rikiav(Vector<Studentas>& A, int rik);
00034 void rikiav(std::list<Studentas>& A, int rik);
00035 void rikiav(std::deque<Studentas>& A, int rik);
00036
00037 // 3 strategijos
00038 void rusiavimas(Vector<Studentas>& A, Vector<Studentas>& vargsai, Vector<Studentas>& kietekai);
00039 void rusiavimas(std::list<Studentas>& A, std::list<Studentas>& vargsai, std::list<Studentas>&
    kietekai);
00040 void rusiavimas(std::deque<Studentas>& A, std::deque<Studentas>& vargsai, std::deque<Studentas>&
    kietekai);
00041
00042 void strat2(Vector<Studentas>& A, Vector<Studentas>& vargsai);
00043 void strat2(std::list<Studentas>& A, std::list<Studentas>& vargsai);
00044 void strat2(std::deque<Studentas>& A, std::deque<Studentas>& vargsai);
00045
00046 void strat3(Vector<Studentas>& A, Vector<Studentas>& vargsai);
00047 void strat3(std::list<Studentas>& A, std::list<Studentas>& vargsai);
00048 void strat3(std::deque<Studentas>& A, std::deque<Studentas>& vargsai);
00049
00050 // tyrimai
00051 void tyrimas1();
00052 void tyrimas2(Vector<Studentas>& A);
00053 void tyrimas2_ve(Vector<Studentas>& A);
00054 void tyrimas2_li(std::list<Studentas>& A);
00055 void tyrimas2_de(std::deque<Studentas>& A);
00056
00057 // testas
00058 void check(bool salyga, const std::string pavad);
00059 void testavimas();
00060 void laiko_test();
00061 void atminties_testas();
00062
00063 #endif

```

## 6.5 header.h File Reference

```

#include <vector>
#include <string>
#include <iostream>
#include "vec.h"

```



## Classes

- class [Zmogus](#)
- class [Studentas](#)

## Functions

- double [mediana](#) ([Vector](#)< double > v)
- double [vidurkis](#) ([Vector](#)< double > v)
- bool [compare](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalEgza](#) (const [Studentas](#) &a, const [Studentas](#) &b)

## 6.5.1 Function Documentation

### 6.5.1.1 [compare\(\)](#)

```
bool compare (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line [85](#) of file [studentas.cpp](#).

### 6.5.1.2 [comparePagalEgza\(\)](#)

```
bool comparePagalEgza (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line [91](#) of file [studentas.cpp](#).

### 6.5.1.3 [comparePagalPavarde\(\)](#)

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line [88](#) of file [studentas.cpp](#).

### 6.5.1.4 [mediana\(\)](#)

```
double mediana (  
    Vector< double > v)
```

Definition at line [64](#) of file [studentas.cpp](#).

### 6.5.1.5 vidurkis()

```
double vidurkis (
    Vector< double > v)
```

Definition at line 76 of file [studentas.cpp](#).

## 6.6 header.h

[Go to the documentation of this file.](#)

```
00001 #ifndef header_H
00002 #define header_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <iostream>
00007 #include "vec.h"
00008
00009 double mediana(Vector<double> v);
00010 double vidurkis(Vector<double> v);
00011
00012 class Zmogus {
00013 protected:
00014     std::string vardas_;
00015     std::string pavarde_;
00016 public:
00017     Zmogus () {}
00018     Zmogus(const std::string& vardas, const std::string& pavarde) : vardas_(vardas), pavarde_(pavarde)
00019     {}
00020     virtual std::string vardas() const {return vardas_;}
00021     virtual std::string pavarde() const {return pavarde_;}
00022
00023     virtual void setVardas(std::string& v){vardas_ = v;}
00024     virtual void setPavarde(std::string& p){pavarde_ = p;}
00025
00026     virtual void spausdinti() const = 0;
00027
00028     virtual ~Zmogus() {
00029         vardas_.clear();
00030         pavarde_.clear();
00031     }
00032
00033 };
00034 class Studentas : public Zmogus {
00035 private:
00036     double egzaminas_;
00037     Vector<double> nd_;
00038 public:
00039     Studentas() : Zmogus(), egzaminas_(0) {}
00040     Studentas(std::istream& is);
00041     // konstruktorius su parametrais
00042     Studentas(const std::string& vardas, const std::string& pavarde, int egzaminas, const
00043     Vector<double>& nd);
00044     // getters
00045     inline int egzaminas() const {return egzaminas_;}
00046     inline Vector<double> nd() const {return nd_;}
00047     // skaiciavimai
00048     double galutBalas(double (*f) (Vector<double>) = mediana) const;
00049     std::istream& readStudent(std::istream& is);
00050     // setters
00051     void setEgz(double egzam){egzaminas_ = egzam;}
00052     void setNd(double x){nd_.push_back(x);}
00053
00054     // ivestis, isvestis
00055     friend std::istream& operator>>(std::istream& is, Studentas& s);
00056     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00057
00058     // rule of five
00059     Studentas(const Studentas& other); // copy constructor
00060
00061     Studentas(Studentas&& other) noexcept; // move constructor
00062
00063     Studentas& operator=(const Studentas& other); // priskyrimo kopijavimo operatorius
00064
00065     Studentas& operator=(Studentas&& other) noexcept; // priskyrimo move operatorius
```

```

00066     void spausdinti() const override;
00067
00068     ~Studentas() {
00069         vardas_.clear();
00070         pavarde_.clear();
00071         nd_.clear();
00072         egzaminas_=0;
00073     } // destructor
00074 };
00075
00076 bool compare(const Studentas& a, const Studentas& b);
00077 bool comparePagalPavarde(const Studentas& a, const Studentas& b);
00078 bool comparePagalEgza(const Studentas& a, const Studentas& b);
00079
00080
00081 #endif

```

## 6.7 README.md File Reference

## 6.8 studentas.cpp File Reference

```

#include "header.h"
#include <algorithm>
#include <sstream>
#include <iomanip>
#include "vec.h"

```

### Functions

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)
- double [mediana](#) ([Vector](#)< double > v)
- double [vidurkis](#) ([Vector](#)< double > v)
- bool [compare](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalEgza](#) (const [Studentas](#) &a, const [Studentas](#) &b)

### 6.8.1 Function Documentation

#### 6.8.1.1 compare()

```

bool compare (
    const Studentas & a,
    const Studentas & b)

```

Definition at line 85 of file [studentas.cpp](#).

#### 6.8.1.2 comparePagalEgza()

```

bool comparePagalEgza (
    const Studentas & a,
    const Studentas & b)

```

Definition at line 91 of file [studentas.cpp](#).

#### 6.8.1.3 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line 88 of file [studentas.cpp](#).

#### 6.8.1.4 mediana()

```
double mediana (  
    Vector< double > v)
```

Definition at line 64 of file [studentas.cpp](#).

#### 6.8.1.5 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s)
```

Definition at line 53 of file [studentas.cpp](#).

#### 6.8.1.6 operator>>()

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

Definition at line 49 of file [studentas.cpp](#).

#### 6.8.1.7 vidurkis()

```
double vidurkis (  
    Vector< double > v)
```

Definition at line 76 of file [studentas.cpp](#).

## 6.9 studentas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "header.h"
00002 #include <algorithm>
00003 #include <sstream>
00004 #include <iomanip>
00005 #include "vec.h"
00006
00007 using std::left;
00008 using std::setw;
00009 using std::fixed;
00010 using std::setprecision;
00011
00012 // konstruktoriaus realizacija
00013 Studentas::Studentas(std::istream& is):Zmogus(), egzaminas_(0) {
00014     readStudent(is);
00015 }
00016
00017 // Studentas::galBalas realizacija
00018 double Studentas::galutBalas(double (*f) (Vector<double>)) const {
00019     return 0.4*f(nd_) + 0.6*egzaminas_;
00020 }
00021
00022 Studentas::Studentas(const std::string& vardas, const std::string& pavarde, int egzaminas, const
    Vector<double>& nd):
00023     Zmogus(vardas,pavarde),
00024     egzaminas_(egzaminas),
00025     nd_(nd) {}
00026
00027
00028 // Studentas::readStudent realizacija
00029 std::istream& Studentas::readStudent(std::istream& is) {
00030     nd_.clear();
00031     is>>vardas_>>pavarde_;
00032
00033     std::string eilute;
00034     std::getline(is, eilute);
00035     std::istringstream buf(eilute);
00036     Vector<double> sk;
00037     int paz;
00038     while(buf>>paz) sk.push_back(paz);
00039
00040     if(!sk.empty()){
00041         egzaminas_=sk.back();
00042         sk.pop_back();
00043         nd_ = sk;
00044     }
00045     return is;
00046 }
00047
00048 // ivestis
00049 std::istream& operator>(std::istream& is, Studentas& s) {
00050     return s.readStudent(is);
00051 }
00052 // isvestis
00053 std::ostream& operator<(std::ostream& os, const Studentas& s) {
00054     os<<left<<setw(20)<<s.pavarde_<<left<<setw(20)<<s.vardas_<<" ND: ";
00055     os<<fixed<<setprecision(2);
00056     for(const auto& x : s.nd_){
00057         os<<x<<" ";
00058     }
00059     os<<"EGZ: " <<s.egzaminas_<<fixed<<setprecision(2)<<" VID: " <<s.galutBalas(vidurkis)<<" MED: "
    <<s.galutBalas();
00060
00061     return os;
00062 }
00063
00064 double mediana(Vector<double> v){
00065     std::sort(v.begin(), v.end());
00066     int n = v.size();
00067
00068     if(n%2){
00069         return v[n/2];
00070     }
00071     else {
00072         return (v[n/2-1]+v[n/2]) /2.0;
00073     }
00074 }
00075
00076 double vidurkis(Vector<double> v){
00077     double s=0;
00078     for(auto x:v){
00079         s+=x;
00080     }

```

```

00081     return s / v.size();
00082 }
00083
00084 //palyginimai
00085 bool compare(const Studentas& a, const Studentas& b){
00086     return a.vardas() < b.vardas();
00087 }
00088 bool comparePagalPavarde(const Studentas& a, const Studentas& b){
00089     return a.pavarde() < b.pavarde();
00090 }
00091 bool comparePagalEgza(const Studentas& a, const Studentas& b){
00092     return a.egzaminas() < b.egzaminas();
00093 }
00094
00095
00096 // Rule of five
00097
00098 //copy constructor
00099 Studentas::Studentas(const Studentas& other):
00100     Zmogus(other.vardas_, other.pavarde_,
00101     egzaminas_{other.egzaminas_},
00102     nd_{std::move(other.nd_)}{})
00103
00104
00105 //move constructor
00106 Studentas::Studentas(Studentas&& other) noexcept:
00107     Zmogus(std::move(other.vardas_), std::move(other.pavarde_),
00108     egzaminas_ (std::move(other.egzaminas_)),
00109     nd_ (std::move(other.nd_)){
00110         other.egzaminas_=0.0;
00111     }
00112
00113
00114 // priskyrimas
00115 Studentas& Studentas::operator=(const Studentas& other){
00116     if(&other == this) return *this;
00117
00118     vardas_=other.vardas_;
00119     pavarde_=other.pavarde_;
00120     egzaminas_=other.egzaminas_;
00121     nd_=other.nd_;
00122
00123     return *this;
00124 }
00125
00126
00127 // priskyrimo move
00128 Studentas& Studentas::operator=(Studentas&& other) noexcept{
00129     if(&other == this ) return *this; //saves priskyrimo tikrinimas
00130
00131     vardas_=std::move(other.vardas_);
00132     pavarde_=std::move(other.pavarde_);
00133     egzaminas_=std::move(other.egzaminas_);
00134     nd_=std::move(other.nd_);
00135
00136     return *this;
00137 }
00138
00139 void Studentas::spausdinti() const {
00140     std::cout<<vardas_<< " "<<pavarde_<<"\n";
00141 }

```

## 6.10 vec.h File Reference

```

#include <vector>
#include <string>
#include <iostream>
#include <limits>
#include <type_traits>
#include <iterator>

```

### Classes

- class [Vector< T, Allocator >](#)

## Functions

- `template<typename U>`  
`bool operator== (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`bool operator!= (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`bool operator< (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`bool operator> (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`bool operator<= (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`bool operator>= (const Vector< U > &a, const Vector< U > &b)`
- `template<typename U>`  
`void swap (Vector< U > &a, Vector< U > &b) noexcept`

## 6.10.1 Function Documentation

### 6.10.1.1 `operator"!=()`

```
template<typename U>
bool operator!= (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 320 of file [vec.h](#).

### 6.10.1.2 `operator<()`

```
template<typename U>
bool operator< (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 325 of file [vec.h](#).

### 6.10.1.3 `operator<=()`

```
template<typename U>
bool operator<= (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 335 of file [vec.h](#).

#### 6.10.1.4 operator==()

```
template<typename U>
bool operator== (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 311 of file [vec.h](#).

#### 6.10.1.5 operator>()

```
template<typename U>
bool operator> (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 330 of file [vec.h](#).

#### 6.10.1.6 operator>=()

```
template<typename U>
bool operator>= (
    const Vector< U > & a,
    const Vector< U > & b)
```

Definition at line 339 of file [vec.h](#).

#### 6.10.1.7 swap()

```
template<typename U>
void swap (
    Vector< U > & a,
    Vector< U > & b) [noexcept]
```

Definition at line 344 of file [vec.h](#).

## 6.11 vec.h

[Go to the documentation of this file.](#)

```
00001 #ifndef vec_H
00002 #define vec_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <iostream>
00007 #include <limits>
00008 #include <type_traits>
00009 #include <iterator>
00010
00011 template<class T, class Allocator=std::allocator<T>
00012 class Vector {
00013 public:
00014     // member types
00015     using value_type = T;
00016     using allocator_type = Allocator;
```



```

00017     using size_type = std::size_t;
00018     using difference_type = std::ptrdiff_t;
00019     using reference = value_type&;
00020     using const_reference = const value_type&;
00021     using pointer = typename std::allocator_traits<Allocator>::pointer;
00022     using const_pointer = typename std::allocator_traits<Allocator>::const_pointer;
00023     using iterator = T*;
00024     using const_iterator = const T*;
00025     using reverse_iterator = std::reverse_iterator<iterator>;
00026     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00027 private:
00028     T* data_;
00029     size_type size_;
00030     size_type capacity_;
00031
00032     void reallocate(size_type new_cap) {
00033         T* new_data = new T[new_cap];
00034         for(size_type i = 0; i < size_; i++) {
00035             new_data[i] = std::move(data_[i]);
00036         }
00037         delete[] data_;
00038         data_ = new_data;
00039         capacity_ = new_cap;
00040     }
00041 public:
00042     // constr
00043     Vector() : data_(nullptr), size_(0), capacity_(0) {}
00044
00045     explicit Vector(size_type n, const T& value = T()): data_(new T[n]), size_(n), capacity_(n) {
00046         for(size_type i = 0; i < n; i++) {
00047             data_[i] = value;
00048         }
00049     }
00050     Vector(std::initializer_list<T> init): data_(new T[init.size()]), size_(init.size()),
00051         capacity_(init.size()) {
00052         size_type i = 0;
00053         for(const auto& v: init) {
00054             data_[i++] = v;
00055         }
00056     }
00057     // RULE OF FIVE -----
00058     ~Vector() {delete[] data_;}
00059     Vector(const Vector& other):
00060         data_(new T[other.capacity_]),
00061         size_(other.size_),
00062         capacity_(other.capacity_) {
00063         std::copy(other.data_, other.data_ + size_, data_);
00064     }
00065
00066     Vector(Vector&& other) noexcept
00067     : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
00068         other.data_ = nullptr;
00069         other.size_ = 0;
00070         other.capacity_ = 0;
00071     }
00072
00073     Vector& operator=(const Vector& other) {
00074         if(this != &other) {
00075             delete[] data_;
00076             size_ = other.size_;
00077             capacity_ = other.capacity_;
00078             data_ = new T[capacity_];
00079             std::copy(other.data_, other.data_ + size_, data_);
00080         }
00081         return *this;
00082     }
00083
00084     Vector& operator=(Vector&& other) noexcept {
00085         if(this != &other) {
00086             delete[] data_;
00087             data_ = other.data_;
00088             size_ = other.size_;
00089             capacity_ = other.capacity_;
00090             other.data_ = nullptr;
00091             other.size_ = 0;
00092             other.capacity_ = 0;
00093         }
00094         return *this;
00095     }
00096
00097     // assign
00098     void assign(size_type count, const T& value) {
00099         if(count > capacity_) {
00100             reserve(count);
00101         }
00102     }

```

```

00103     }
00104     for(size_type i = 0; i<count; i++){
00105         data_[i] = value;
00106     }
00107     size_ = count;
00108 }
00109
00110 template<typename InputIt>
00111 std::enable_if_t<!std::is_integral<InputIt>::value, void>
00112 assign(InputIt first, InputIt last){
00113     clear();
00114     for(; first != last; ++first){
00115         push_back(*first);
00116     }
00117 }
00118
00119 template<typename Range>
00120 void assign_range(const Range& r){
00121     assign(r.begin(), r.end());
00122 }
00123
00124 // - get allocator
00125
00126 std::allocator<T> get_allocator() const{
00127     return std::allocator<T>();
00128 }
00129
00130
00131 // element access
00132
00133 reference at(size_type i ){
00134     if(i >= size_) throw std::out_of_range("Vector::at");
00135     return data_[i];
00136 }
00137
00138 const_reference at(size_type i) const {
00139     if(i >= size_) throw std::out_of_range("Vector::at");
00140     return data_[i];
00141 }
00142
00143 reference operator[](size_type i){ return data_[i]; }
00144 const_reference operator[](size_type i) const {return data_[i];}
00145
00146 reference front() {return data_[0];}
00147 const_reference front() const {return data_[0];}
00148
00149 reference back() {return data_[size_ -1];}
00150 const_reference back() const {return data_[size_ -1];}
00151
00152 T* data() {return data_;}
00153 const T* data() const {return data_;}
00154
00155
00156 // iterators
00157
00158 iterator begin() { return data_; }
00159 const_iterator begin() const { return data_; }
00160
00161 iterator end() { return data_ + size_; }
00162 const_iterator end() const { return data_ + size_; }
00163
00164 const_iterator cbegin() const { return data_; }
00165 const_iterator cend() const { return data_ + size_; }
00166
00167 reverse_iterator rbegin() { return reverse_iterator(end());}
00168 reverse_iterator rend() { return reverse_iterator(begin());}
00169
00170 const_reverse_iterator crbegin() const { return const_reverse_iterator(end());}
00171 const_reverse_iterator crend() const { return const_reverse_iterator(begin());}
00172
00173
00174 // capacity
00175
00176 bool empty() const { return size_ == 0; }
00177
00178 size_type size() const { return size_; }
00179 size_type max_size() const { return std::numeric_limits<size_type>::max(); }
00180
00181 void reserve(size_type new_cap){
00182     if(new_cap > capacity_)
00183         reallocate(new_cap);
00184 }
00185
00186 size_type capacity() const { return capacity_; }
00187
00188 void shrink_to_fit(){
00189     if(size_ < capacity_){

```

```

00190         reallocate(size_);
00191     }
00192 }
00193
00194
00195 // modifiers
00196
00197 void clear() { size_ = 0; }
00198
00199 iterator insert(iterator pos, const T& value){
00200     size_type index = pos - begin();
00201     if(size_ == capacity_){
00202         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00203     }
00204     for(size_type i = size_; i > index; i--){
00205         data_[i] = std::move(data_[i-1]);
00206     }
00207     data_[index] = value;
00208     ++size_;
00209     return begin() + index;
00210 }
00211
00212 iterator insert(iterator pos, size_type count, const T& value){
00213     size_type index = pos - begin();
00214     while(size_ + count > capacity_){
00215         reserve(capacity_ == 0 ? count : capacity_ * 2);
00216     }
00217     for(size_type i = size_; i > index; i--){
00218         data_[i+count-1] = std::move(data_[i-1]);
00219     }
00220     for(size_type i=0; i< count; i++){
00221         data_[index+i]=value;
00222     }
00223     size_ += count;
00224     return begin() + index;
00225 }
00226
00227 template<typename... Args>
00228 iterator emplace(iterator pos, Args&&... args){
00229     size_type index = pos - begin();
00230     if(size_ == capacity_){
00231         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00232     }
00233     for(size_type i = size_; i > index; i--){
00234         data_[i] = std::move(data_[i-1]);
00235     }
00236     data_[index] = T(std::forward<Args>(args)...);
00237     ++size_;
00238     return begin() + index;
00239 }
00240
00241 iterator erase(iterator pos){
00242     size_type index = pos - begin();
00243     for(size_type i = index; i<size_-1; i++){
00244         data_[i] = std::move(data_[i+1]);
00245     }
00246     --size_;
00247     return begin() + index;
00248 }
00249
00250 iterator erase(iterator first, iterator last){
00251     size_type start = first - begin();
00252     size_type count = last - first;
00253     for(size_type i = start; i + count < size_; i++){
00254         data_[i] = std::move(data_[i+count]);
00255     }
00256     size_ -= count;
00257     return begin() + start;
00258 }
00259
00260 void push_back(const T& value){
00261     if(size_ == capacity_){
00262         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00263     }
00264     data_[size_++] = value;
00265 }
00266
00267 template<typename... Args>
00268 reference emplace_back(Args&&... args){
00269     if(size_ == capacity_){
00270         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00271     }
00272     data_[size_] = T(std::forward<Args>(args)...);
00273     return data_[size_++];
00274 }
00275
00276 template<typename Range>

```

```

00277 void append_range(const Range& r){
00278     for(const auto& elem : r ){
00279         push_back(elem);
00280     }
00281 }
00282
00283 void pop_back(){
00284     if(size_>0){
00285         --size_;
00286     }
00287 }
00288
00289 void resize(size_type new_size){
00290     if(new_size > capacity_)
00291         reallocate(new_size);
00292
00293     for(size_type i = size_; i < new_size; ++i)
00294         data_[i]=T();
00295
00296     size_ = new_size;
00297 }
00298
00299 void swap(Vector& other) noexcept {
00300     std::swap(data_, other.data_);
00301     std::swap(size_, other.size_);
00302     std::swap(capacity_, other.capacity_);
00303 }
00304
00305 };
00306
00307
00308 // non-member
00309
00310 template<typename U>
00311 bool operator==(const Vector<U>& a, const Vector<U>& b){
00312     if(a.size() != b.size()) return false;
00313     for(std::size_t i=0; i < a.size(); i++){
00314         if(a[i] != b[i]) return false;
00315     }
00316     return true;
00317 }
00318
00319 template<typename U>
00320 bool operator!=(const Vector<U>& a, const Vector<U>& b){
00321     return !(a==b);
00322 }
00323
00324 template<typename U>
00325 bool operator<(const Vector<U>& a, const Vector<U>& b){
00326     return std::lexicographical_compare(a.begin(), a.end(), b.begin(), b.end());
00327 }
00328
00329 template<typename U>
00330 bool operator>(const Vector<U>& a, const Vector<U>& b){
00331     return b < a;
00332 }
00333
00334 template<typename U>
00335 bool operator<=(const Vector<U>& a, const Vector<U>& b){
00336     return !(b < a);
00337 }
00338 template<typename U>
00339 bool operator>=(const Vector<U>& a, const Vector<U>& b){
00340     return !(a < b);
00341 }
00342
00343 template<typename U>
00344 void swap(Vector<U>& a, Vector<U>& b) noexcept {
00345     a.swap(b);
00346 }
00347
00348 #endif

```

## 6.12 vector.cpp File Reference

```

#include <vector>
#include <iomanip>
#include <iostream>
#include <algorithm>

```

```
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <windows.h>
#include <list>
#include <deque>
#include "funkcijos.h"
#include "vec.h"
```

## Typedefs

- typedef std::uniform\_int\_distribution< int > [int\\_dis](#)

## Functions

- int [main](#) ()

### 6.12.1 Typedef Documentation

#### 6.12.1.1 int\_dis

```
typedef std::uniform_int_distribution<int> int\_dis
```

Definition at line [27](#) of file [vector.cpp](#).

### 6.12.2 Function Documentation

#### 6.12.2.1 main()

```
int main ()
```

Definition at line [29](#) of file [vector.cpp](#).

## 6.13 vector.cpp

[Go to the documentation of this file.](#)

```

00001 #include <vector>
00002 #include <iomanip>
00003 #include <iostream>
00004 #include <algorithm>
00005 #include <random>
00006 #include <chrono>
00007 #include <limits>
00008 #include <fstream>
00009 #include <windows.h>
00010 #include <list>
00011 #include <deque>
00012 #include "funkcijos.h"
00013 #include "vec.h"
00014
00015 using std::cout;
00016 using std::cin;
00017 using std::endl;
00018 using std::setw;
00019 using std::left;
00020 using std::mt19937;
00021 using std::setprecision;
00022 using std::fixed;
00023 using std::string;
00024 using std::getline;
00025
00026 using laik = std::chrono::high_resolution_clock;
00027 typedef std::uniform_int_distribution<int> int_dis;
00028
00029 int main() {
00030     SetConsoleOutputCP(CP_UTF8);
00031     SetConsoleCP(CP_UTF8);
00032
00033     int x;
00034     long n;
00035     long m;
00036     int meniu;
00037     Vector<Studentas> A;
00038     std::list<Studentas> L;
00039     std::deque<Studentas> D;
00040
00041     Studentas tmp;
00042     while(true) {
00043         cout<< "1 - ranka\n"
00044             << "2 - generuoti pažymius\n"
00045             << "3 - generuoti pažymius, vardus, pavardes\n"
00046             << "4 - nuskaityti iš failo\n"
00047             << "5 - generuoti failus\n"
00048             << "6 - testas 1 \n"
00049             << "7 - testas 2 \n"
00050             << "8 - testas 2 su vector\n"
00051             << "9 - testas 2 su list\n"
00052             << "10 - testas 2 su deque\n"
00053             << "11 - rule of five testavimas\n"
00054             << "12 - vector laiko testavimas\n"
00055             << "13 - perskirstymų testavimas\n"
00056             << "14 - baigti darbą" << endl;
00057         while(true) {
00058             cin>>meniu;
00059             if(cin.fail() || meniu<1 || meniu>14) {
00060                 cin.clear();
00061                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00062                 cout<<"Įvesti galima tik skaičius nuo 1 iki 14" << endl;
00063             }
00064             else if (cin.peek() != ' ' && cin.peek() != '\n') {
00065                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00066                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 14" << endl;
00067             }
00068             else {
00069                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00070                 break;
00071             }
00072
00073             if(meniu==1) {
00074                 A.clear();
00075                 while(true) {
00076                     tmp = Studentas();
00077                     string tvardas;
00078                     string tpavarde;
00079
00080                     cout<<"Įvesk studento vardą arba paspausk ENTER, jei nebeiori įvesti daugiau
studentų" << endl;
00081                     getline(cin, tvardas);

```

```

00082         if(tvardas.empty()){break;}
00083         tmp.setVardas(tvardas);
00084
00085         cout<<"Įvesk studento pavardę"<<endl;
00086         getline(cin, tpavarde);
00087         tmp.setPavarde(tpavarde);
00088
00089         cout<<"Įvesk pažymį nuo 1 iki 10 arba 0, jei nori baigti įvedimą"<<endl;
00090         while(true){
00091             cin>>x;
00092             if(cin.fail()){
00093                 cin.clear();
00094                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00095                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10 arba 0"<<endl;
00096                 continue;
00097             }
00098             else if(cin.peek() != ' ' && cin.peek() != '\n'){
00099                 cin.ignore(10000, '\n');
00100                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00101                 continue;
00102             }
00103             else if(x==0){break;
00104                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00105             }
00106
00107             else if(x<1 || x>10){
00108                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00109                 continue;
00110             }
00111             tmp.setNd(x);
00112         }
00113         cout<<"Įvesk egzamino pažymį nuo 1 iki 10"<<endl;
00114         while (true){
00115             int tegz;
00116             cin>>tegz;
00117             if(cin.fail()|| tegz<1 || tegz>10){
00118                 cin.clear();
00119                 cin.ignore(10000, '\n');
00120                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00121             }
00122             else if(cin.peek() != ' ' && cin.peek() != '\n'){
00123                 cin.ignore(10000, '\n');
00124                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00125             }
00126             else{
00127                 tmp.setEgz(tegz);
00128                 break;}
00129         }
00130         A.push_back(tmp);
00131         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00132     }
00133     isved(A);
00134 }
00135
00136 else if(meniu==2){
00137     A.clear();
00138     cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00139     while(true){
00140         tmp = Studentas();
00141         string tvardas;
00142         string tpavarde;
00143
00144         cout<<"Įvesk studento vardą arba paspausk ENTER, jei nebenori įvesti daugiau
studentų"<<endl;
00145         getline(cin, tvardas);
00146         if(tvardas.empty()){break;}
00147         tmp.setVardas(tvardas);
00148
00149         cout<<"Įvesk studento pavardę"<<endl;
00150         getline(cin, tpavarde);
00151         tmp.setPavarde(tpavarde);
00152
00153         cout<<"Įvesk kiek pažymių gavo už namų darbus"<<endl;
00154         while(true){
00155             cin>>n;
00156             if(cin.fail()||n<=0){
00157                 cin.clear();
00158                 cin.ignore(10000, '\n');
00159                 cout<<"Įvesti galima tik sveikuosius teigiamus skaičius"<<endl;
00160             }
00161             else if(cin.peek() != ' ' && cin.peek() != '\n'){
00162                 cin.ignore(10000, '\n');
00163                 cout<<"Įvesti galima tik sveikuosius skaičius"<<endl;
00164             }
00165             else{break;}
00166         }
00167     }

```

```

00168         for(long j=0; j<n; j++){
00169             tmp.setNd(gen_pazym());
00170         }
00171         tmp.setEgz(gen_pazym());
00172         A.push_back(tmp);
00173         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00174     }
00175     isved(A);
00176 }
00177
00178 else if(meniu==3){
00179     A.clear();
00180     skaiciai(n,m);
00181     A.reserve(m);
00182     for(int i=0; i<m; i++){
00183         Studentas temp = gen_vrd();
00184         for(int j=0; j<n; j++){
00185             temp.setNd(gen_pazym());
00186         }
00187         temp.setEgz(gen_pazym());
00188         A.push_back(temp);
00189     }
00190     isved(A);
00191 }
00192
00193 else if (meniu == 4){
00194     A.clear();
00195     try{
00196         skt(A, "studentai10000.txt");
00197     }
00198     catch(const std::exception& e){
00199         std::cerr<<"Klaida : " <<e.what()<<endl;
00200         return 1;
00201     }
00202     int rik = pasirink();
00203     rikiav(A, rik);
00204
00205     int m2;
00206     cout<<"1 - Išvesti į ekraną, 2 - Išvesti į failą"<<endl;
00207     while(true){
00208         cin>>m2;
00209         if(cin.fail() || m2<1 || m2>2){
00210             cin.clear();
00211             cin.ignore(10000, '\n');
00212             cout<<"Išvesti galima tik 1, arba 2"<<endl;
00213         }
00214         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00215             cin.ignore(10000, '\n');
00216             cout<<"Išvesti galima tik sveikuosius skaičius 1, arba 2"<<endl;
00217         }
00218         else{break;}
00219     }
00220
00221     if(m2==1){
00222         //isved(A);
00223         cout<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis
(Med.)"<<endl;
00225         cout<<"-----"<<endl;
00226         for(const auto& s: A){
00227             cout<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<setw(17)<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<<setprecision(2)<<s.galutMed()<<endl;
00228         }
00229     }
00230     else if(m2==2){
00231         // ofstream
00232         std::ofstream r("rezultatai.txt");
00233         r<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis
(Med.)"<<endl;
00234         r<<"-----"<<endl;
00235         for(const auto& s: A){
00236             r<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<setw(17)<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<<setprecision(2)<<s.galutMed()<<endl;
00237         }
00238         r.close();
00239     }
00240 }
00241
00242 else if(meniu==5){
00243     failu_kurimas();
00244 }
00245 else if(meniu==6){
00246     tyrimas1();
00247 }
00248 else if(meniu==7){
00249     A.clear();
00250     tyrimas2(A);

```



```
00251     }
00252     else if(meniu==8){
00253         A.clear();
00254         // su vektoriais
00255         tyrimas2_ve(A);
00256     }
00257     else if(meniu==9){
00258         A.clear();
00259         // su list
00260         std::list<Studentas> AL;
00261         tyrimas2_li(AL);
00262     }
00263     else if(meniu==10){
00264         A.clear();
00265         // su deque
00266         std::deque<Studentas> AD;
00267         tyrimas2_de(AD);
00268     }
00269     else if(meniu==11){
00270         testavimas();
00271     }
00272     else if(meniu==12){
00273         laiko_test();
00274     }
00275     else if(meniu==13){
00276         atminties_testas();
00277     }
00278     else if(meniu==14){
00279         break;
00280     }
00281 }
00282 }
```



# Index

- ~Studentas
  - Studentas, [11](#)
- ~Vector
  - Vector< T, Allocator >, [17](#)
- ~Zmogus
  - Zmogus, [25](#)
- allocator\_type
  - Vector< T, Allocator >, [14](#)
- append\_range
  - Vector< T, Allocator >, [17](#)
- assign
  - Vector< T, Allocator >, [17](#)
- assign\_range
  - Vector< T, Allocator >, [18](#)
- at
  - Vector< T, Allocator >, [18](#)
- atminties\_testas
  - funkcijos.cpp, [29](#)
  - funkcijos.h, [42](#)
- back
  - Vector< T, Allocator >, [18](#)
- begin
  - Vector< T, Allocator >, [18](#), [19](#)
- capacity
  - Vector< T, Allocator >, [19](#)
- cbegin
  - Vector< T, Allocator >, [19](#)
- cend
  - Vector< T, Allocator >, [19](#)
- check
  - funkcijos.cpp, [29](#)
  - funkcijos.h, [42](#)
- Class, [1](#)
- clear
  - Vector< T, Allocator >, [19](#)
- compare
  - header.h, [49](#)
  - studentas.cpp, [51](#)
- comparePagalEgza
  - header.h, [49](#)
  - studentas.cpp, [51](#)
- comparePagalPavarde
  - header.h, [49](#)
  - studentas.cpp, [51](#)
- const\_iterator
  - Vector< T, Allocator >, [14](#)
- const\_pointer
  - Vector< T, Allocator >, [15](#)
- const\_reference
  - Vector< T, Allocator >, [15](#)
- const\_reverse\_iterator
  - Vector< T, Allocator >, [15](#)
- crbegin
  - Vector< T, Allocator >, [19](#)
- crend
  - Vector< T, Allocator >, [19](#)
- data
  - Vector< T, Allocator >, [20](#)
- difference\_type
  - Vector< T, Allocator >, [15](#)
- egzaminas
  - Studentas, [11](#)
- emplace
  - Vector< T, Allocator >, [20](#)
- emplace\_back
  - Vector< T, Allocator >, [20](#)
- empty
  - Vector< T, Allocator >, [20](#)
- end
  - Vector< T, Allocator >, [20](#), [21](#)
- erase
  - Vector< T, Allocator >, [21](#)
- failu\_kurimas
  - funkcijos.cpp, [29](#)
  - funkcijos.h, [43](#)
- front
  - Vector< T, Allocator >, [21](#)
- funkcijos.cpp, [27](#)
  - atminties\_testas, [29](#)
  - check, [29](#)
  - failu\_kurimas, [29](#)
  - gen\_pazym, [29](#)
  - gen\_vrd, [29](#)
  - int\_dis, [28](#)
  - isved, [29](#)
  - isvedimas\_faila, [29](#), [30](#)
  - isvedimas\_faila\_visi, [30](#)
  - laik, [28](#)
  - laiko\_test, [30](#)
  - pasirink, [30](#)
  - rikiav, [30](#), [31](#)
  - rikiav\_visi, [31](#)
  - rusiavimas, [31](#)
  - rusiavimas\_visi, [31](#)

- skaiciai, 32
- skt, 32
- skt\_visi, 32
- strat2, 32, 33
- strat2\_visi, 33
- strat3, 33
- strat3\_visi, 34
- testavimas, 34
- tyrimas1, 34
- tyrimas2, 34
- tyrimas2\_de, 34
- tyrimas2\_li, 34
- tyrimas2\_ve, 34
- tyrimas2\_visi, 35
- funkcijos.h, 41
  - atminties\_testas, 42
  - check, 42
  - failu\_kurimas, 43
  - gen\_pazym, 43
  - gen\_vrd, 43
  - isved, 43
  - isvedimas\_faila, 43
  - laiko\_test, 44
  - pasirink, 44
  - rikiav, 44
  - rusiavimas, 44, 45
  - skaiciai, 45
  - skt, 45
  - strat2, 45, 46
  - strat3, 46
  - testavimas, 46
  - tyrimas1, 47
  - tyrimas2, 47
  - tyrimas2\_de, 47
  - tyrimas2\_li, 47
  - tyrimas2\_ve, 47
- galutBalas
  - Studentas, 11
- gen\_pazym
  - funkcijos.cpp, 29
  - funkcijos.h, 43
- gen\_vrd
  - funkcijos.cpp, 29
  - funkcijos.h, 43
- get\_allocator
  - Vector< T, Allocator >, 21
- header.h, 48
  - compare, 49
  - comparePagalEgza, 49
  - comparePagalPavarde, 49
  - mediana, 49
  - vidurkis, 49
- insert
  - Vector< T, Allocator >, 22
- int\_dis
  - funkcijos.cpp, 28
- vector.cpp, 61
- isved
  - funkcijos.cpp, 29
  - funkcijos.h, 43
- isvedimas\_faila
  - funkcijos.cpp, 29, 30
  - funkcijos.h, 43
- isvedimas\_faila\_visi
  - funkcijos.cpp, 30
- iterator
  - Vector< T, Allocator >, 15
- laik
  - funkcijos.cpp, 28
- laiko\_test
  - funkcijos.cpp, 30
  - funkcijos.h, 44
- main
  - vector.cpp, 61
- max\_size
  - Vector< T, Allocator >, 22
- mediana
  - header.h, 49
  - studentas.cpp, 52
- nd
  - Studentas, 11
- operator!=
  - vec.h, 55
- operator<
  - vec.h, 55
- operator<<
  - Studentas, 12
  - studentas.cpp, 52
- operator<=
  - vec.h, 55
- operator>
  - vec.h, 56
- operator>>
  - Studentas, 12
  - studentas.cpp, 52
- operator>=
  - vec.h, 56
- operator=
  - Studentas, 11
  - Vector< T, Allocator >, 22
- operator==
  - vec.h, 55
- operator[]
  - Vector< T, Allocator >, 22, 23
- pasirink
  - funkcijos.cpp, 30
  - funkcijos.h, 44
- pavarde
  - Zmogus, 25
- pavarde\_

- Zmogus, 26
- pointer
  - Vector< T, Allocator >, 15
- pop\_back
  - Vector< T, Allocator >, 23
- push\_back
  - Vector< T, Allocator >, 23
- rbegin
  - Vector< T, Allocator >, 23
- README.md, 51
- readStudent
  - Studentas, 12
- reference
  - Vector< T, Allocator >, 15
- rend
  - Vector< T, Allocator >, 23
- reserve
  - Vector< T, Allocator >, 23
- resize
  - Vector< T, Allocator >, 24
- reverse\_iterator
  - Vector< T, Allocator >, 16
- rikiav
  - funkcijos.cpp, 30, 31
  - funkcijos.h, 44
- rikiav\_visi
  - funkcijos.cpp, 31
- rusiavimas
  - funkcijos.cpp, 31
  - funkcijos.h, 44, 45
- rusiavimas\_visi
  - funkcijos.cpp, 31
- setEgz
  - Studentas, 12
- setNd
  - Studentas, 12
- setPavarde
  - Zmogus, 25
- setVardas
  - Zmogus, 26
- shrink\_to\_fit
  - Vector< T, Allocator >, 24
- size
  - Vector< T, Allocator >, 24
- size\_type
  - Vector< T, Allocator >, 16
- skaiciai
  - funkcijos.cpp, 32
  - funkcijos.h, 45
- skt
  - funkcijos.cpp, 32
  - funkcijos.h, 45
- skt\_visi
  - funkcijos.cpp, 32
- spausdinti
  - Studentas, 12
  - Zmogus, 26
- strat2
  - funkcijos.cpp, 32, 33
  - funkcijos.h, 45, 46
- strat2\_visi
  - funkcijos.cpp, 33
- strat3
  - funkcijos.cpp, 33
  - funkcijos.h, 46
- strat3\_visi
  - funkcijos.cpp, 34
- Studentas, 9
  - ~Studentas, 11
  - egzaminas, 11
  - galutBalas, 11
  - nd, 11
  - operator<<, 12
  - operator>>, 12
  - operator=, 11
  - readStudent, 12
  - setEgz, 12
  - setNd, 12
  - spausdinti, 12
  - Studentas, 10, 11
- studentas.cpp, 51
  - compare, 51
  - comparePagalEgza, 51
  - comparePagalPavarde, 51
  - mediana, 52
  - operator<<, 52
  - operator>>, 52
  - vidurkis, 52
- swap
  - vec.h, 56
  - Vector< T, Allocator >, 24
- testavimas
  - funkcijos.cpp, 34
  - funkcijos.h, 46
- tyrimas1
  - funkcijos.cpp, 34
  - funkcijos.h, 47
- tyrimas2
  - funkcijos.cpp, 34
  - funkcijos.h, 47
- tyrimas2\_de
  - funkcijos.cpp, 34
  - funkcijos.h, 47
- tyrimas2\_li
  - funkcijos.cpp, 34
  - funkcijos.h, 47
- tyrimas2\_ve
  - funkcijos.cpp, 34
  - funkcijos.h, 47
- tyrimas2\_visi
  - funkcijos.cpp, 35
- value\_type
  - Vector< T, Allocator >, 16
- vardas

- Zmogus, 26
- vardas\_
  - Zmogus, 26
- vec.h, 54
  - operator!=, 55
  - operator<, 55
  - operator<=, 55
  - operator>, 56
  - operator>=, 56
  - operator==, 55
  - swap, 56
- Vector
  - Vector< T, Allocator >, 16, 17
- Vector< T, Allocator >, 13
  - ~Vector, 17
  - allocator\_type, 14
  - append\_range, 17
  - assign, 17
  - assign\_range, 18
  - at, 18
  - back, 18
  - begin, 18, 19
  - capacity, 19
  - cbegin, 19
  - cend, 19
  - clear, 19
  - const\_iterator, 14
  - const\_pointer, 15
  - const\_reference, 15
  - const\_reverse\_iterator, 15
  - crbegin, 19
  - crend, 19
  - data, 20
  - difference\_type, 15
  - emplace, 20
  - emplace\_back, 20
  - empty, 20
  - end, 20, 21
  - erase, 21
  - front, 21
  - get\_allocator, 21
  - insert, 22
  - iterator, 15
  - max\_size, 22
  - operator=, 22
  - operator[], 22, 23
  - pointer, 15
  - pop\_back, 23
  - push\_back, 23
  - rbegin, 23
  - reference, 15
  - rend, 23
  - reserve, 23
  - resize, 24
  - reverse\_iterator, 16
  - shrink\_to\_fit, 24
  - size, 24
  - size\_type, 16
  - swap, 24
  - value\_type, 16
  - Vector, 16, 17
- vector.cpp, 60
  - int\_dis, 61
  - main, 61
- vidurkis
  - header.h, 49
  - studentas.cpp, 52
- Zmogus, 24
  - ~Zmogus, 25
  - pavarde, 25
  - pavarde\_, 26
  - setPavarde, 25
  - setVardas, 26
  - spausdinti, 26
  - vardas, 26
  - vardas\_, 26
  - Zmogus, 25